



**VietNam National University
University of Engineering and Technology**

EMBEDDED SYSTEM FUNDAMENTALS

(ELT3240, NHẬP MÔN HỆ THỐNG NHÚNG)

Dr. Nguyễn Kiêm Hùng

Email: kiemhung@vnu.edu.vn



**Introduction to
Embedded Systems**

**Week
1-2**

**Introduction to C
You Are
Here.**

**Week
3**

**CPU:
ARM Cortex-M**

**Week
4**

**Memory
and Interfaces**

**Week
5-6**

**ARM-based
Embedded System**

**Week
7**

Embedded Software

**Week
8-9**

**Real-time
Operating systems**

**Week
10-12**

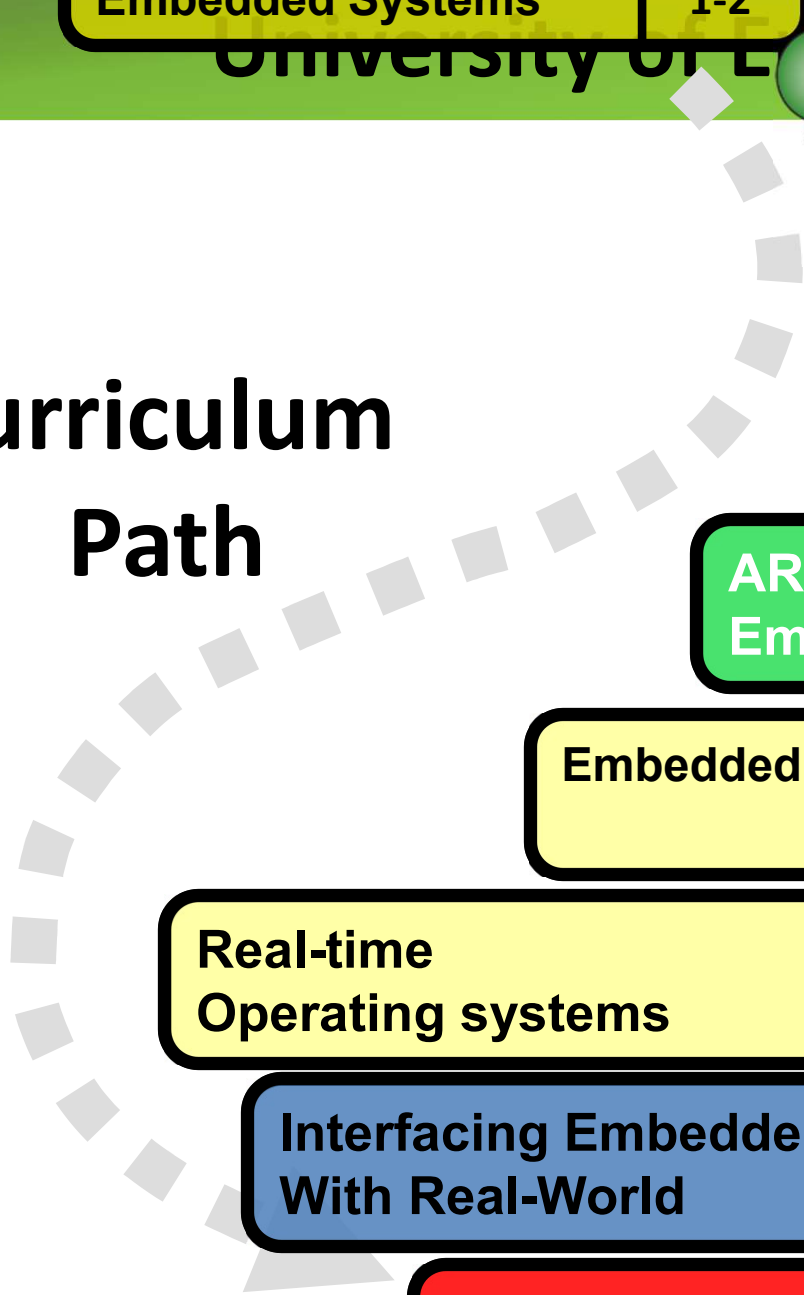
**Interfacing Embedded
With Real-World**

**Week
13-14**

Project

**Week
15**

**Curriculum
Path**



Outline

- Basic Processor Architecture
- Computer architecture taxonomy
- Basic concept of Instruction set



- **ARM architecture**

- ARM versions.

- ARM programming model.

- ARM memory organization.

- ARM Instruction Set

- ARM data processing operations.

- ARM flow control operations.

- Summary



Smartphones

Mobile Internet Devices

Full Internet

Social Networking

Personal Media Player

Web Applications

Notebooks



Nokia N95



Apple iPod Touch



Nokia N810



Intelligent in your pocket!



土豆网 Tudou.com
每个人都是生活的导演

web2.0
workgroup

MID are changing your life...

1,900,000,000 ARM Powered® processors in mobile devices in 2007



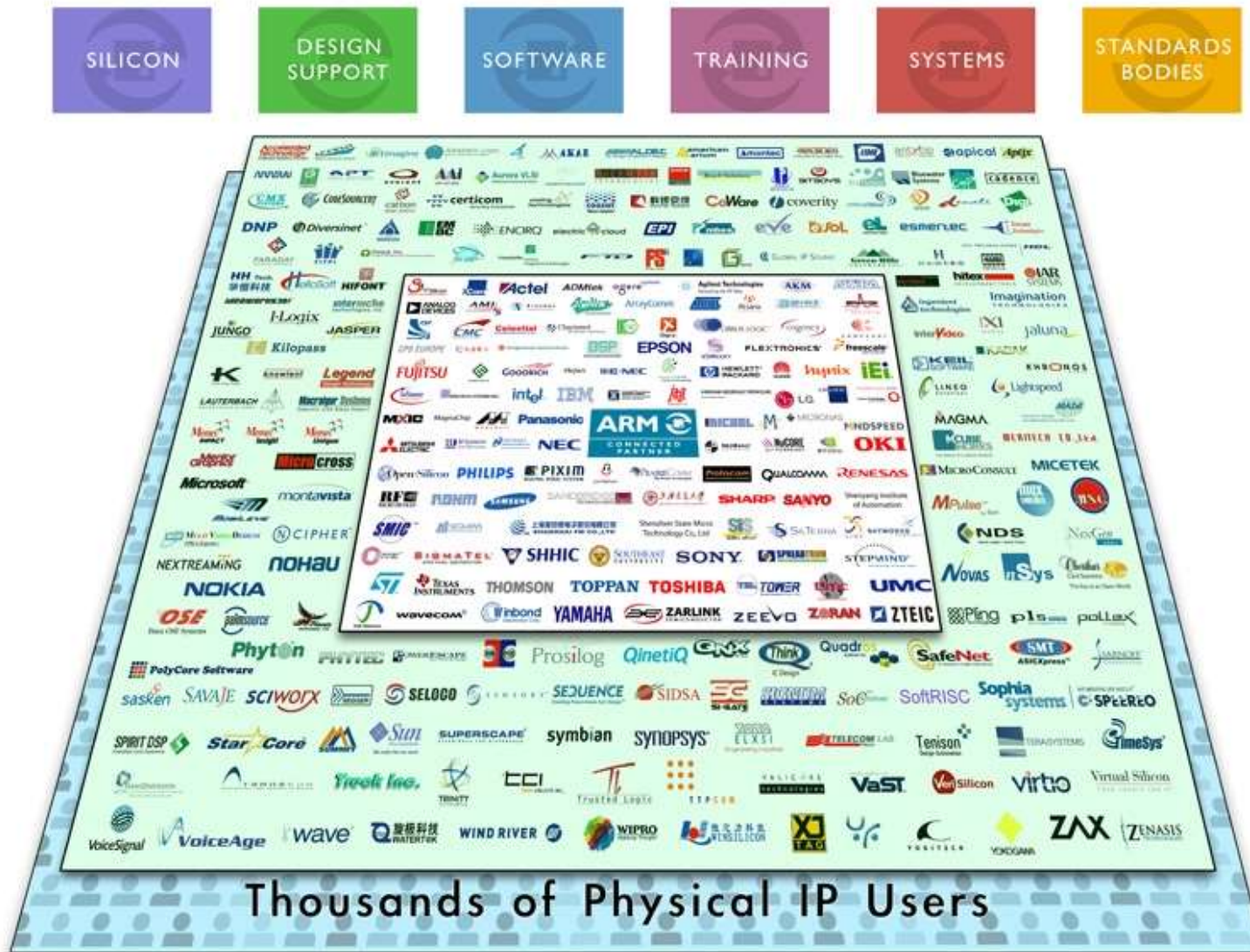
Since 2012, virtually all PDAs and smartphones contain ARM CPUs, and
ARMs account for 75 percent of all 32-bit embedded systems and 90
percent of embedded RISC systems!



ARM

at the heart ... of wireless evolution

ARM Doesn't Actually Produce Microprocessors



Which architecture is my processor?

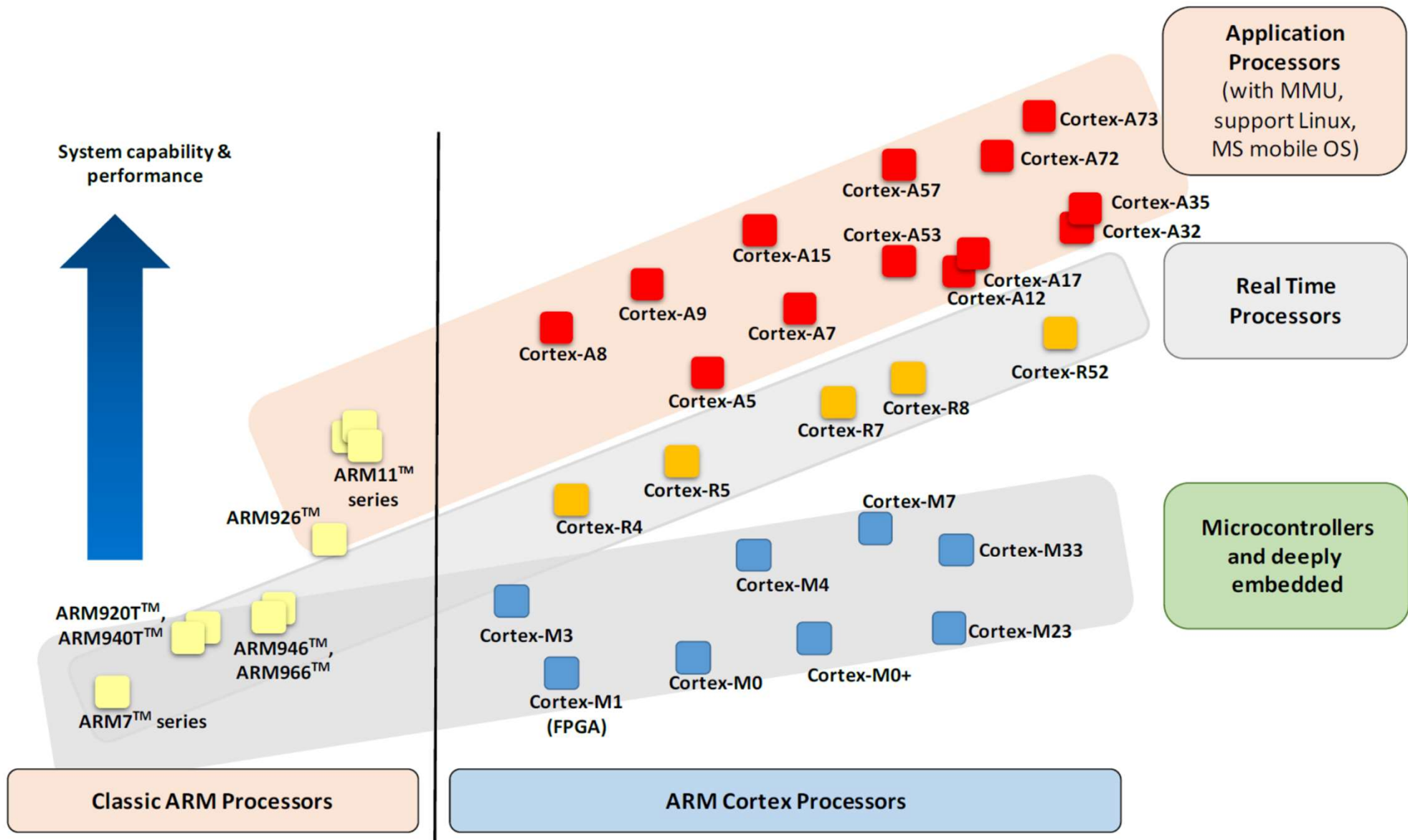
What are differences between ARM7 and ARMv7?

- An ***architecture*** is a design that define the programmer's model, covering all aspects of the design
 - registers, addressing modes, memory architecture, basic operations, etc.
 - modifications from previous architectures
- A ***processor*** is a device that is silicon implementation of architecture
 - depends on an architecture but also adds other features (e.g. pipeline stages, cache size)

Which architecture is my processor?

Processor core	Architecture
• ARM7TDMI family – ARM720T, ARM740T	v4T
• ARM9TDMI family – ARM920T, ARM922T, ARM940T	v4T
• ARM9E family – ARM946E-S, ARM966E-S, ARM926EJ-S	v5TE, v5TEJ
• ARM10E family – ARM1020E, ARM1022E, ARM1026EJ-S	v5TE, v5TEJ
• ARM11 family – ARM1136J(F)-S – ARM1156T2(F)-S – ARM1176JZ(F)-S	v6 v6T2 v6Z
• Cortex family (2004) – ARM Cortex-A8 – ARM Cortex-R4 – ARM Cortex-M3	v7A v7R v7M
■ Note: Implementations of the same architecture can be very different	
■ ARM7TDMI - arch v4T. Von Neuman core with 3 stage pipeline	
■ ARM920T - arch v4T. Harvard core with 5 stage pipeline and MMU	

ARM processor family



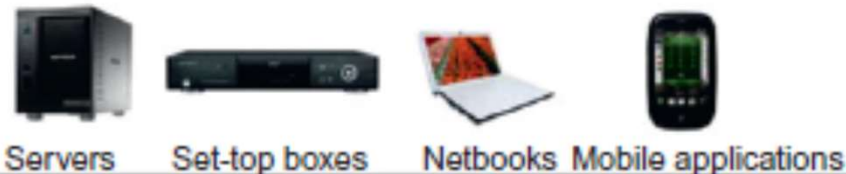
ARM Cortex Profile

The ARM Cortex family includes processors based on the three distinct profiles of the ARMv7 architecture.

- The A profile for sophisticated, high-end **Applications** running open and complex operating systems
- The R profile for **Real-time** systems
- The M profile optimized for cost-sensitive and **Microcontroller** applications



Cortex-A



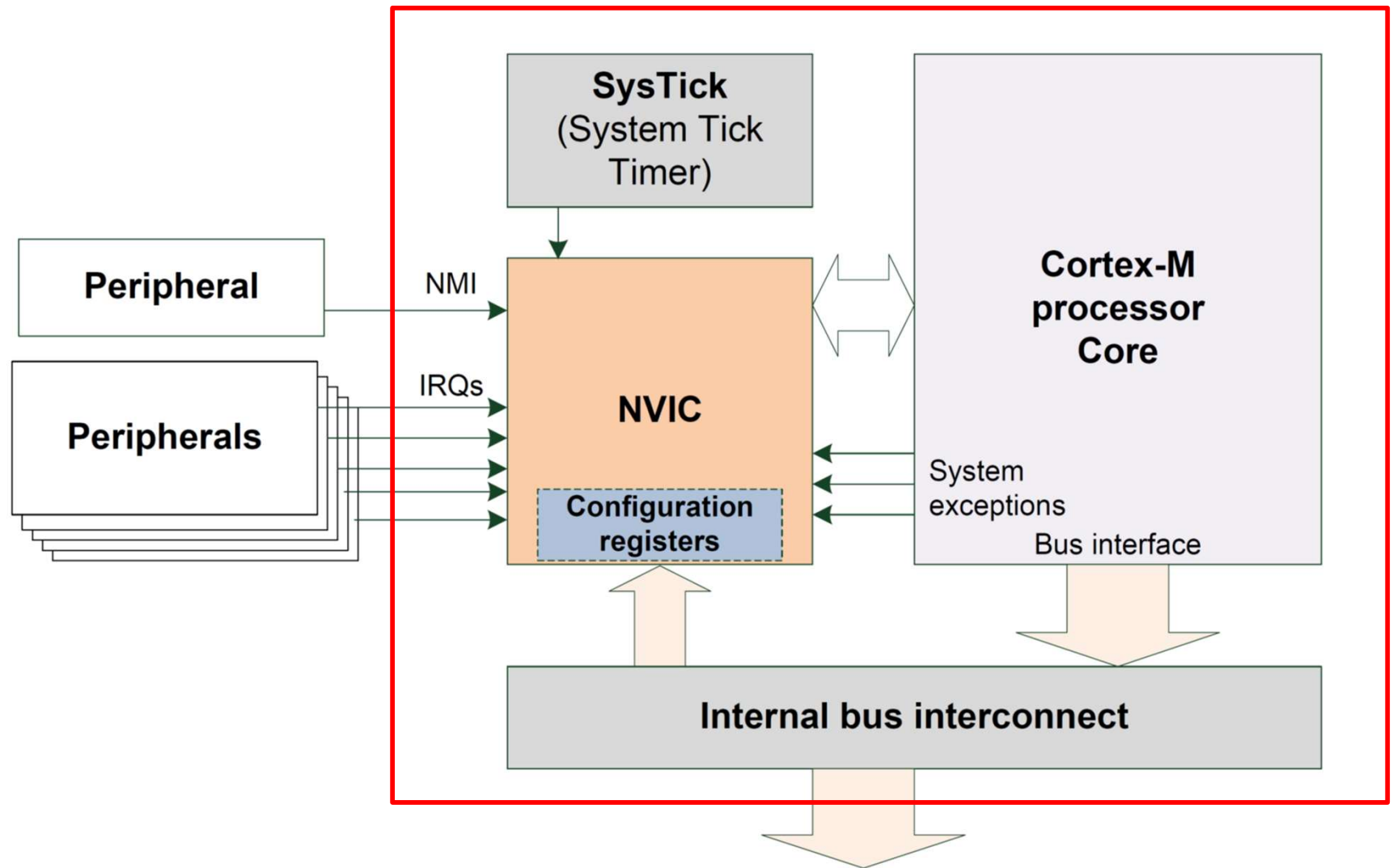
Cortex-R



Cortex-M

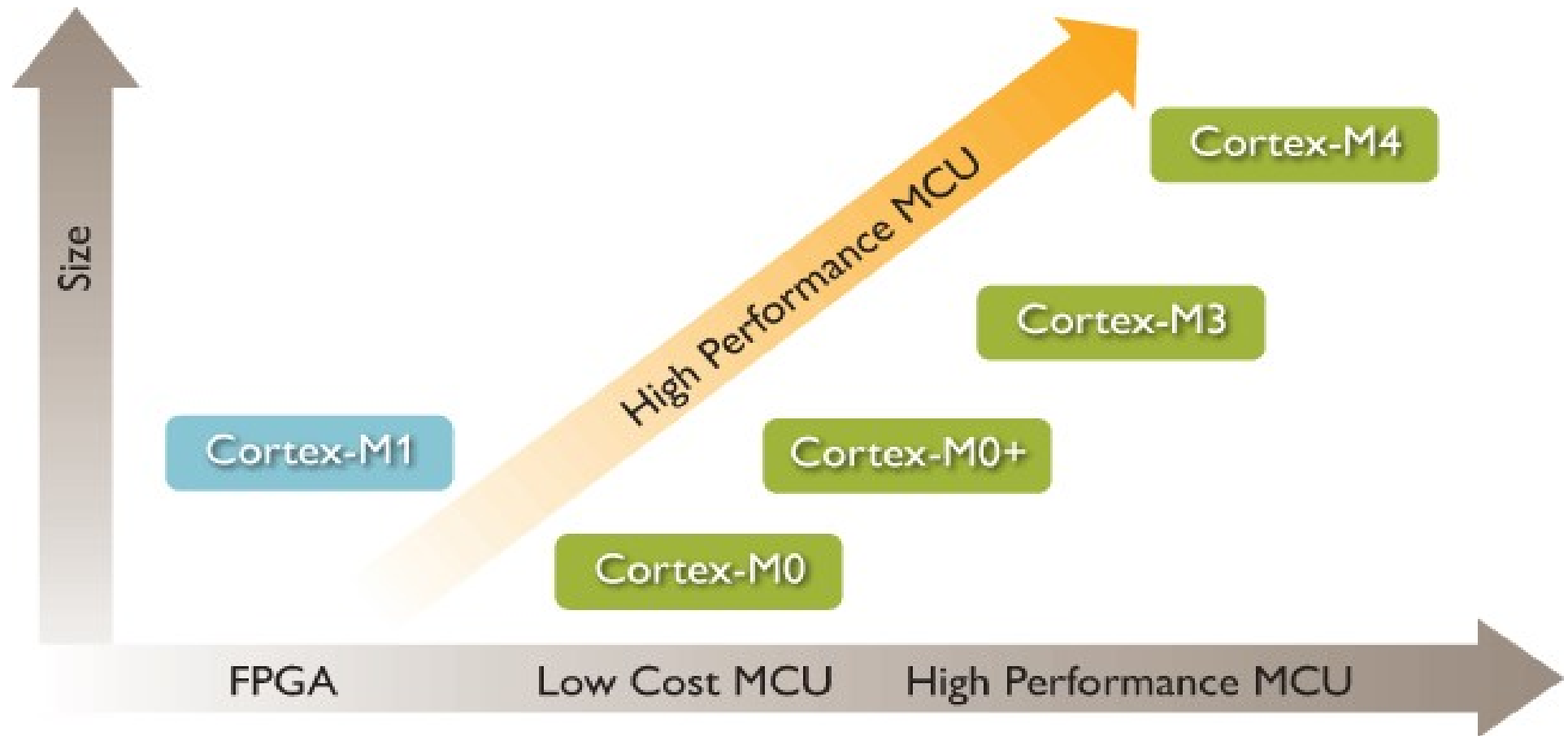


ARM Cortex-M Series



Cortex-M is a complete Microcontroller Unit (MCU) architecture, not just a CPU core.

ARM Cortex-M Series

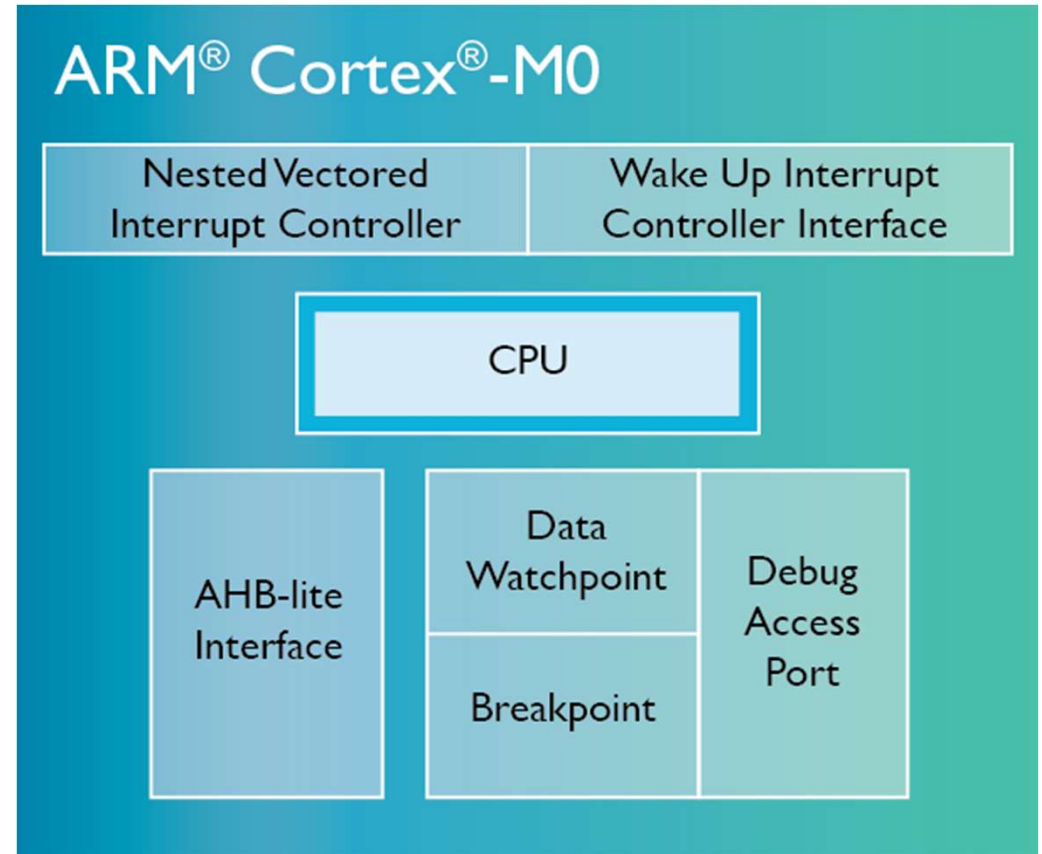


The most common used five different variants of Cortex-M profile

ARM Cortex-M Series

Cortex M0

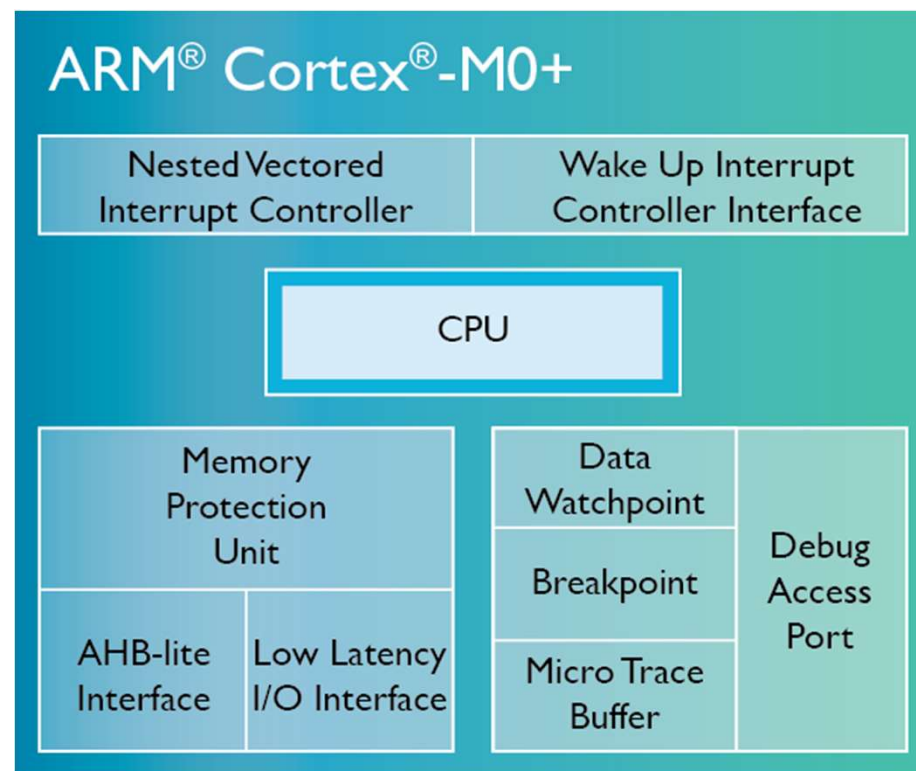
- 32-bit RISC processor
- **3-stage pipeline von Neumann architecture**
- **ARMv6-M architecture**
- 16-bit Thumb instruction set with Thumb-2 technology.
- Load-Store Architecture
- 56 Assembly Instructions
- Low power support



ARM Cortex-M Series

Cortex M0+

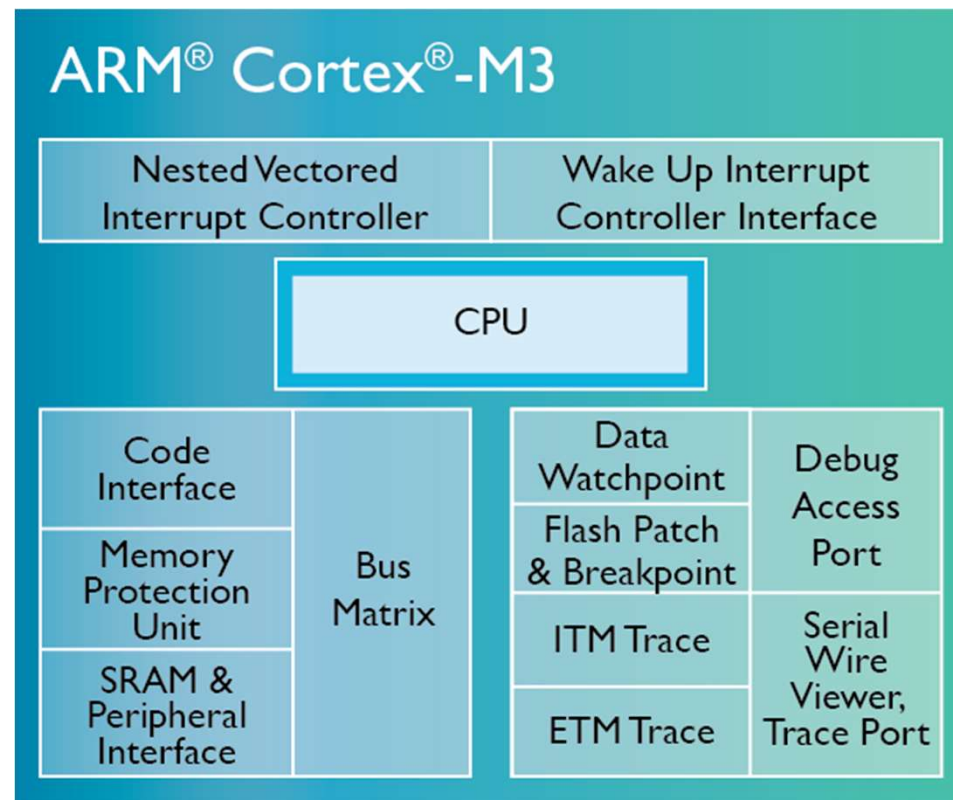
- 32-bit RISC processor
- **2-stage pipeline** enabling faster branch instruction execution with fewer clock cycles and minimizes power consumption
- **ARMv6-M architecture**
- **The most energy efficient ARM processor**
- Cortex-M compatibility – 100% compatible with Cortex-M0 instruction set and a subset the Cortex-M3/M4 instruction set
- Ultra-Low power



ARM Cortex-M Series

Cortex M3

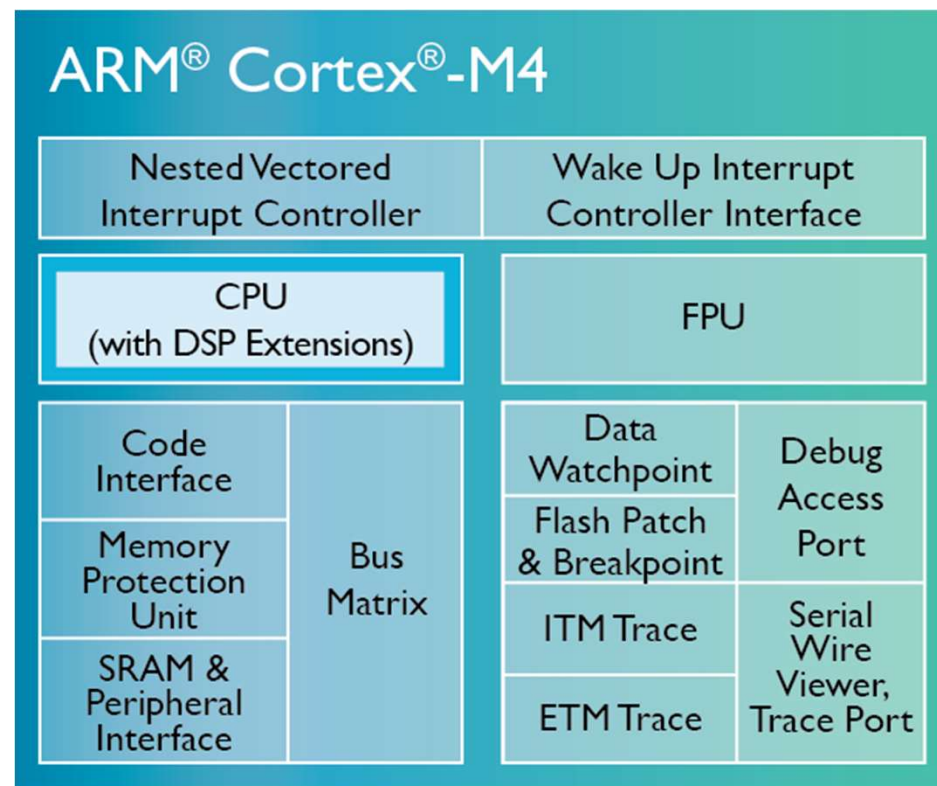
- 32-bit RISC processor
- 3-stage pipeline for high performance embedded system while providing low power advantages.
- **ARMv7-M architecture**
- Thumb[®] /Thumb-2 instruction set.
- High-performance low-cost platforms



ARM Cortex-M Series

Cortex M4

- 32-bit RISC processor
- 3-stage pipeline for high performance embedded system while providing low power advantages.
- **ARMv7-M architecture**
- Thumb® /Thumb-2 instruction set.
- Combination of high-efficiency signal processing functionality with the low-power

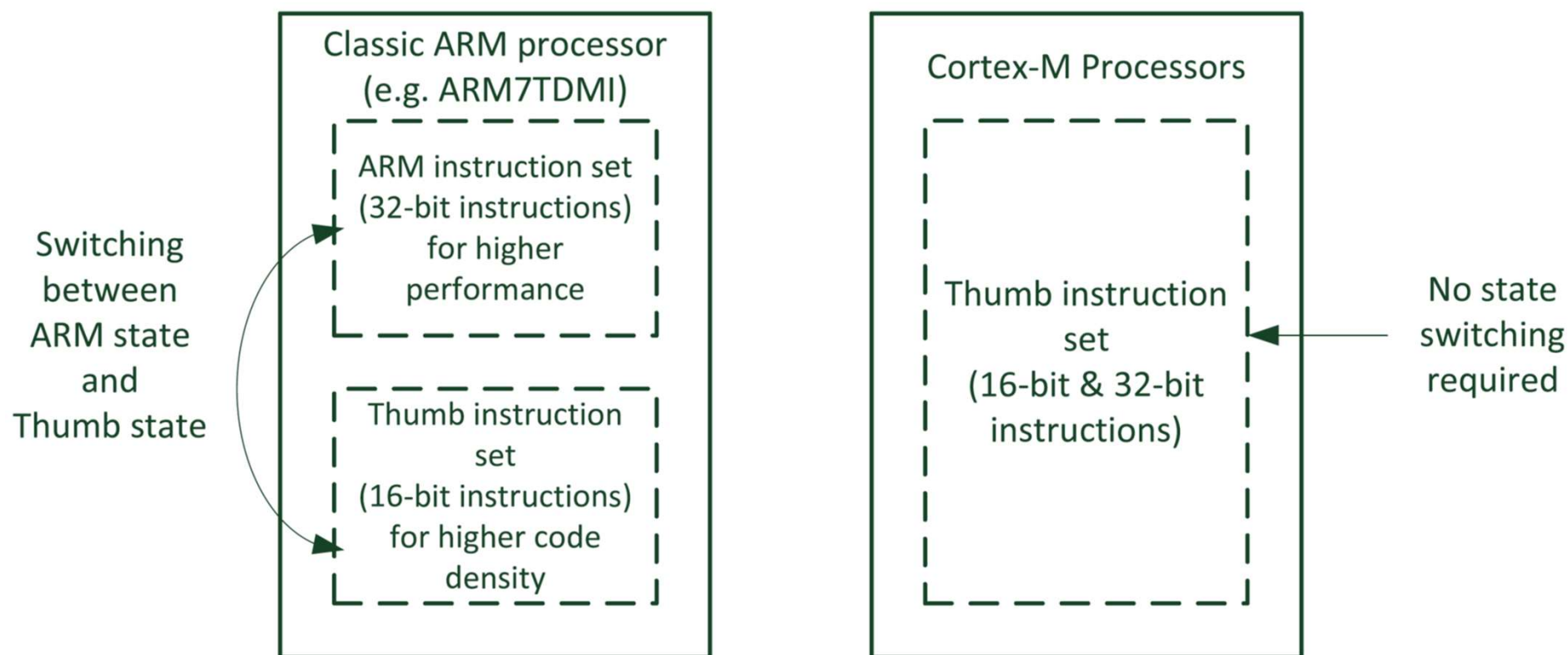


ARM Cortex-M Series

- ARM is a RISC architecture
 - Most instructions execute in a single cycle
- ARM is a 32-bit load / store architecture
 - **operations cannot be performed directly on memory locations**
 - operands must be loaded into the CPU and results are stored back to main memory
- When used in relation to the ARM:
 - **Halfword** means 16 bits (two bytes)
 - **Word** means 32 bits (four bytes)
 - **Doubleword** means 64 bits (eight bytes)
- The earlier ARMs implement two instruction sets:
 - 32-bit **ARM** Instruction Set
 - 16-bit **Thumb** Instruction Set
- Latest ARM cores (e.g. Cortex-M) introduce a new instruction set called **Thumb-2**
 - Provides a mixture of 32-bit and 16-bit instructions
 - Maintains code density with increased flexibility

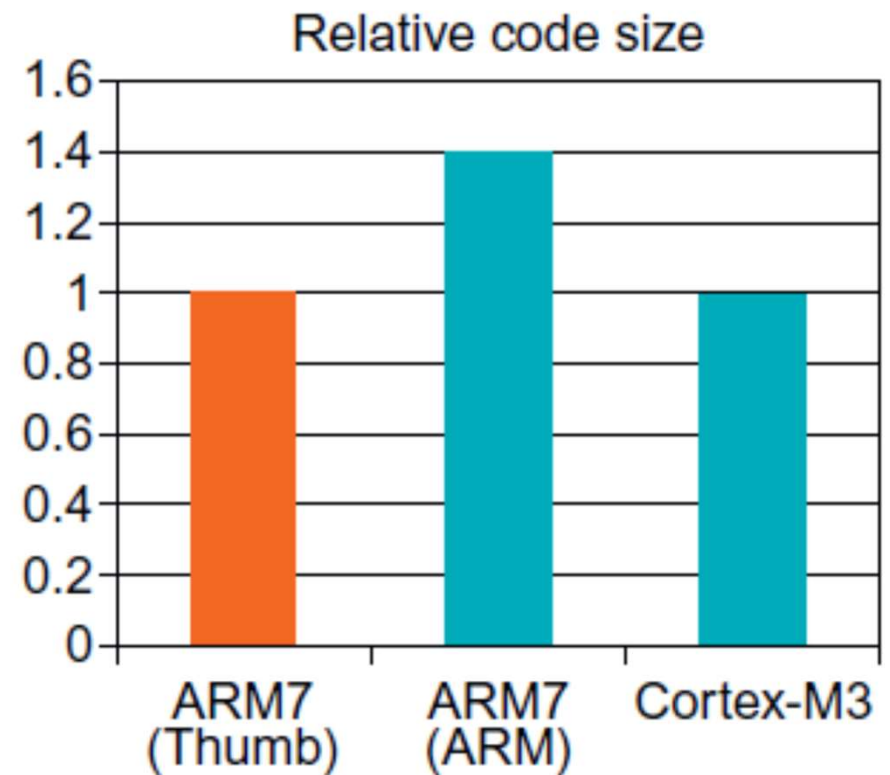
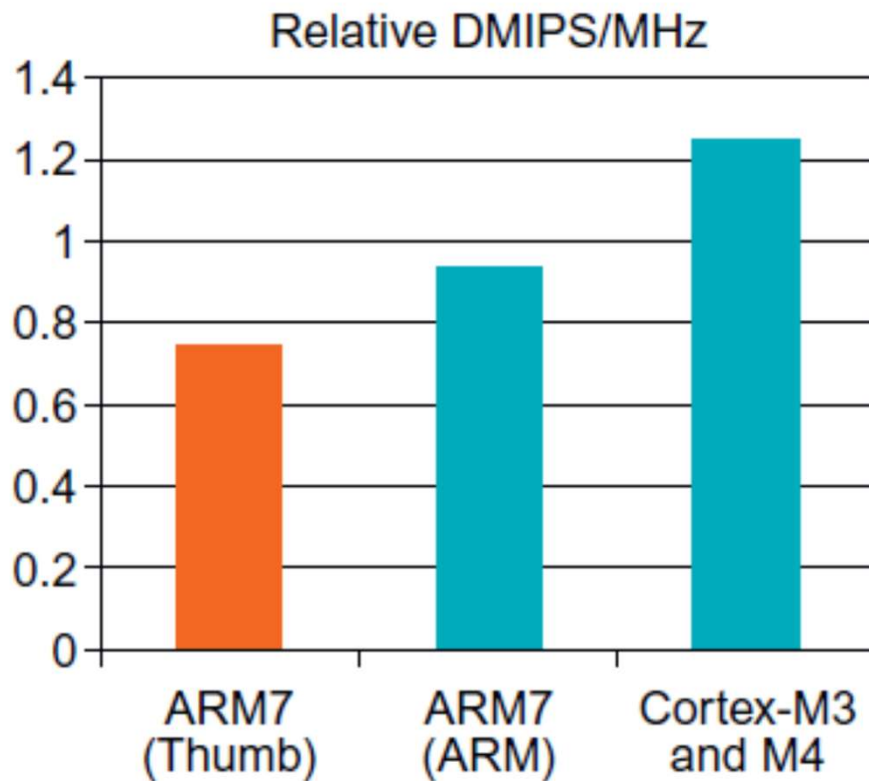
ARM Cortex-M Series

Instruction set comparison between Cortex-M processors and ARM7TDMI:



ARM Cortex-M Series

Instruction set comparison between Cortex-M processors and ARM7TDMI:



ARM Cortex-M Series

Comparing 16-bit Multiply Operations Across Processor Architectures

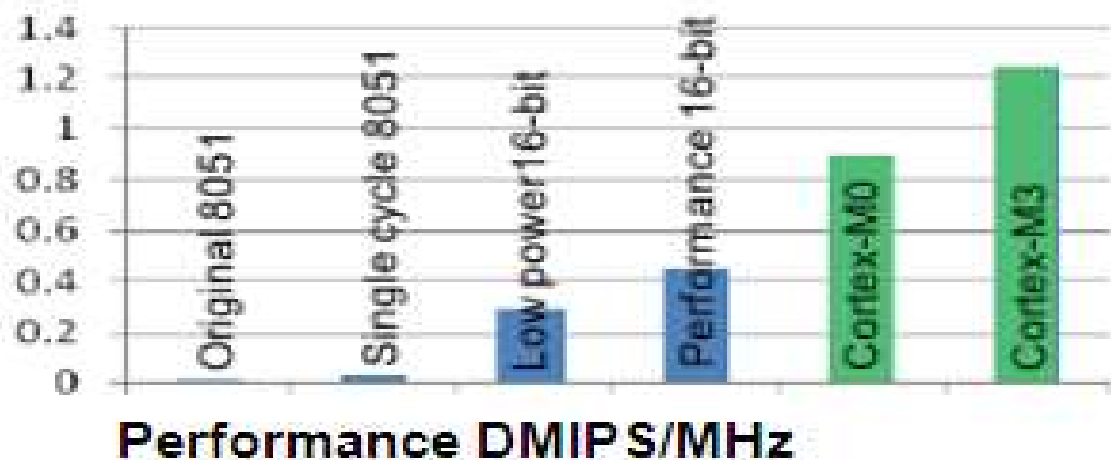
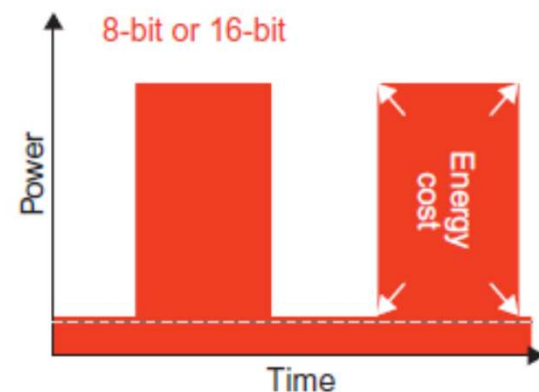
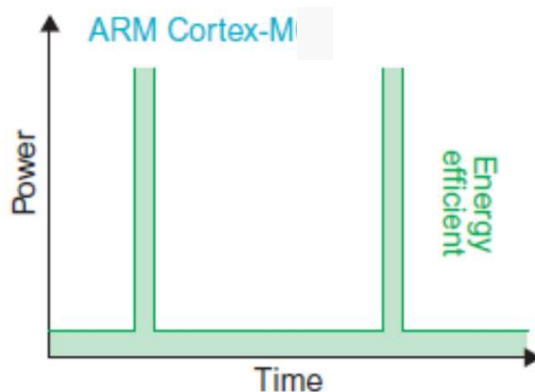
8-Bit Example (8051)	16-Bit Example	ARM Cortex-M
<pre> MOV A, XL ; 2 bytes MOV B, YL ; 3 bytes MUL AB; 1 byte MOV R0, A; 1 byte MOV R1, B; 3 bytes MOV A, XL ; 2 bytes MOV B, YH ; 3 bytes MUL AB; 1 byte ADD A, R1; 1 byte MOV R1, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R2, A; 1 byte MOV A, XH ; 2 bytes MOV B, YL ; 3 bytes MUL AB; 1 byte ADD A, R1; 1 byte MOV R2, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R3, A; 1 byte </pre>	<pre> MOV R1, &MulOp1 MOV R2, &MulOp2 MOV SumLo, R3 MOV SumHi, R4 </pre> (Memory mapped multiply unit)	<pre> MULS r0, r1, r0 </pre>
Time: 48 clock cycles* Code size: 48 bytes	Time: 8 clock cycles Code size: 8 bytes	Time: 1 clock cycle Code size: 2 bytes

cycle count for a single cycle 8051 processor.

ARM Cortex-M Series

Cortex-M Advantages:

1. Energy efficiency
2. Smaller code
3. Ease of use
4. High performance



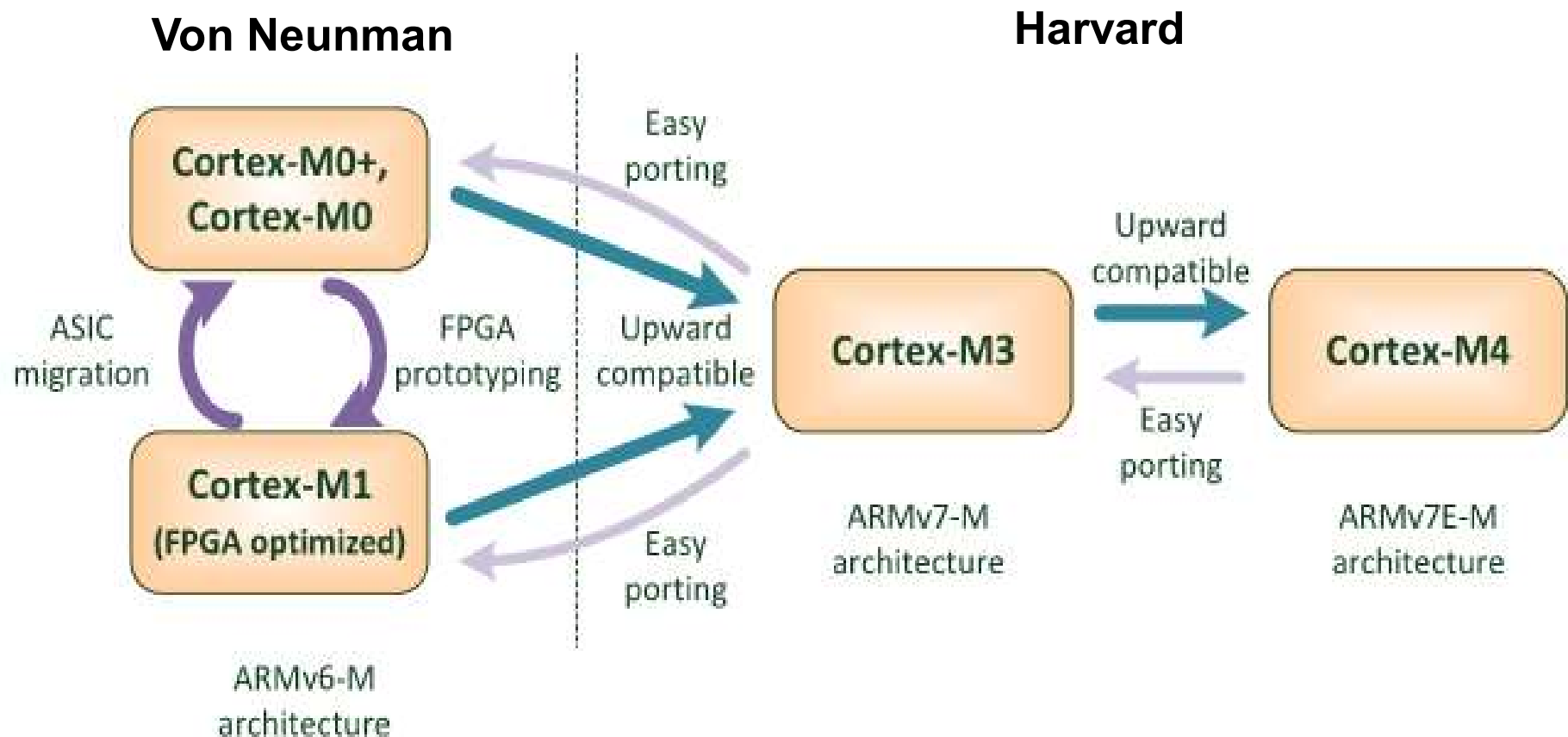
ARM Cortex-M Series

Range of Instructions in Different Cortex-M

Instruction Groups	Cortex-M0, M1	Cortex-M3	Cortex-M4	Cortex-M4 with FPU
16-bit ARMv6-M instructions	●	●	●	●
32-bit Branch with Link instruction	●	●	●	●
32-bit system instructions	●	●	●	●
16-bit ARMv7-M instructions		●	●	●
32-bit ARMv7-M instructions		●	●	●
DSP extensions			●	●
Floating point instructions				●

ARM Cortex-M Series

Compatibility between Cortex-M processors



Cortex-M compatibility

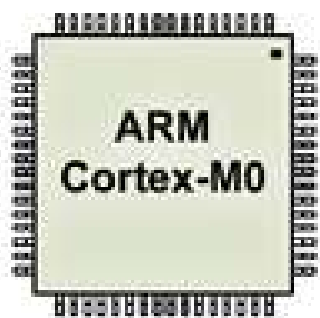
ARM Cortex-M Series



How about Compatibility between the Cortex-M0 and Cortex-M3 processors ?

switching the device driver library

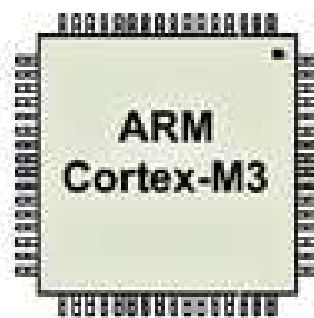
ARMv6-M



0.9 DMIPS/MHz

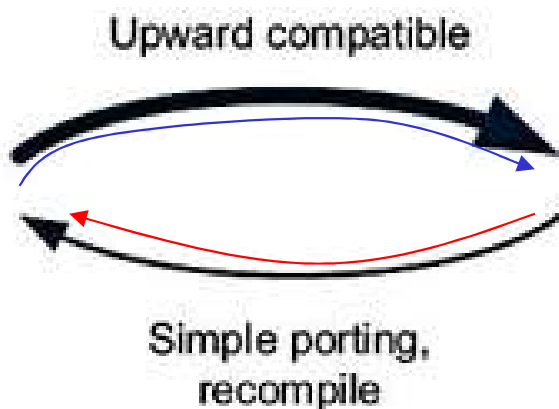
Von Neumann architecture

ARMv7-M



1.25 DMIPS/MHz

Harvard bus architecture



- **NVIC** (Nested Vectored Interrupt Controller) **and SCB** (System Control Block) registers:
 - AVAILABILITY?
 - **8/16-bit** TRANSFER MODES.
- **Bit-band feature**
- **Assembly Instructions**

Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM architecture**
 - ARM versions.
 - **ARM programming model.**
 - ARM memory organization.
 - ARM Instruction Set
 - ARM data processing operations.
 - ARM flow control operations.
- Summary

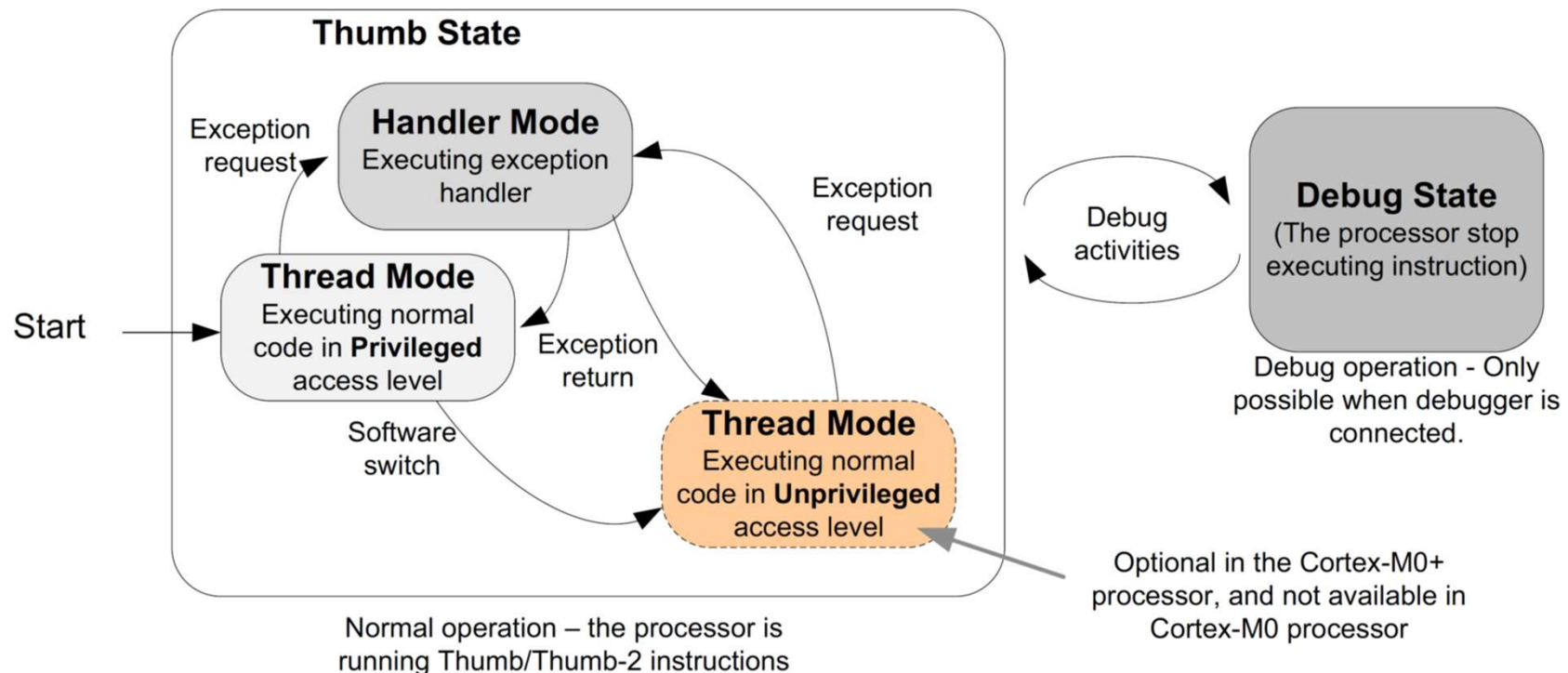


ARM data types

- Word is 32 bits long.
- Word can be divided into four 8-bit bytes.
- ARM addresses can be 32 bits long.
- An address refers to a byte, not a word, in memory.
 - Word 0 is at address of byte 0, word 1 is at address of byte 4, etc.
- ARM processor can be configured at power-up as either **little-endian** or **big-endian** mode.

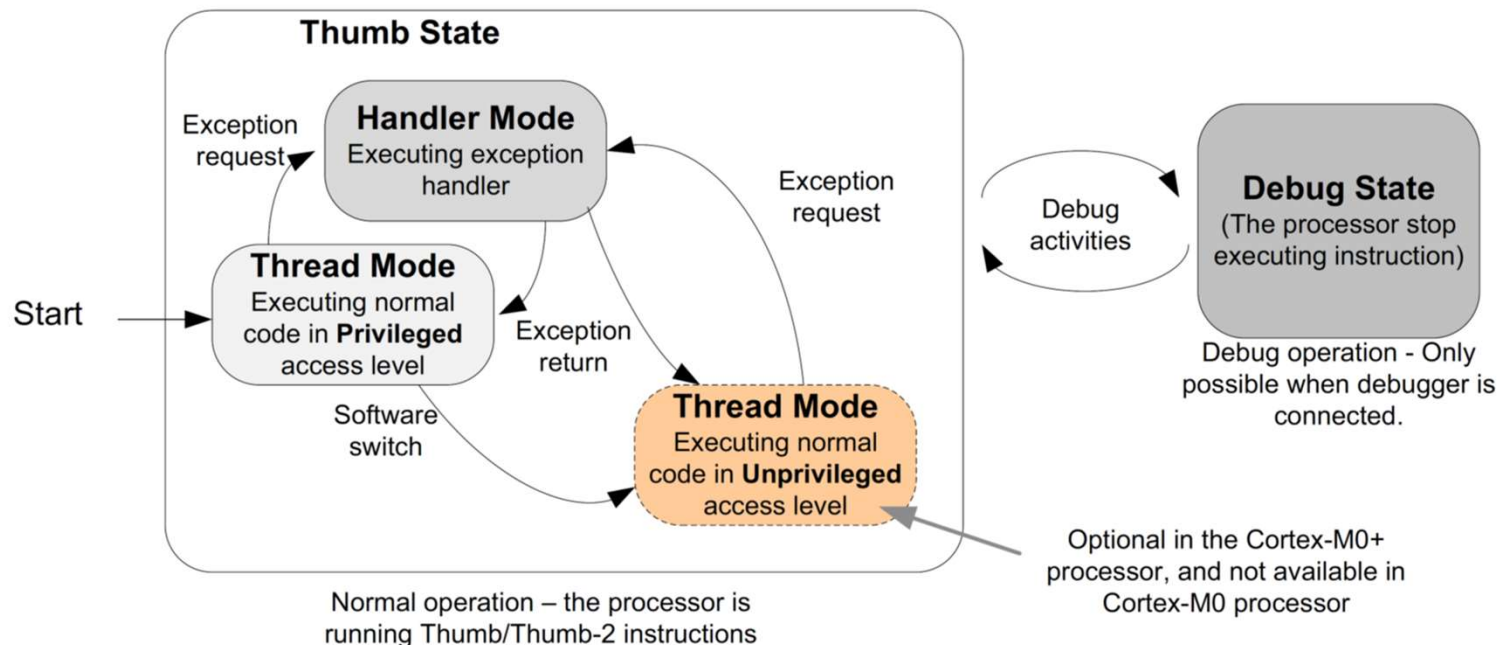
Operation Modes & States

- **Thread mode:**
 - Executes application software. The processor enters Thread mode when it comes out of reset.
- **Handler mode:**
 - Handles exceptions. The processor returns to Thread mode when it has finished all exception processing.



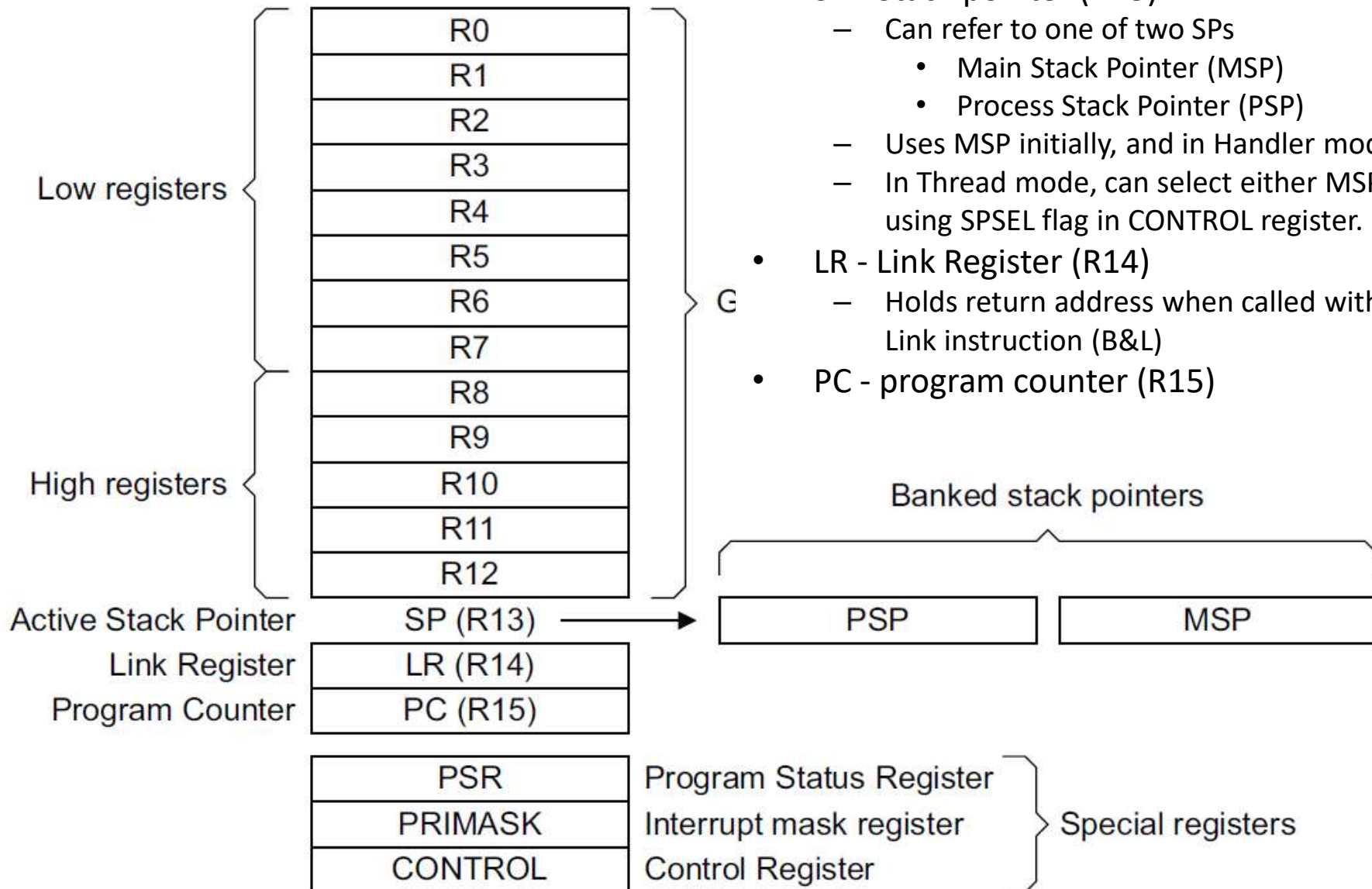
Operation Modes & States

- **Unprivileged access level: The software**
 - has limited access to system registers using the MSR and MRS instructions, and cannot use the CPS instruction to mask interrupts
 - cannot access the system timer, NVIC, or system control block
 - might have restricted access to memory or peripherals.
- **Privileged:**
 - The software can use all the instructions and has access to all resources.



Core Registers

- R0-R12 - General purpose, for data processing
- SP - Stack pointer (R13)
 - Can refer to one of two SPs
 - Main Stack Pointer (MSP)
 - Process Stack Pointer (PSP)
 - Uses MSP initially, and in Handler mode
 - In Thread mode, can select either MSP or PSP using SPSEL flag in CONTROL register.
- LR - Link Register (R14)
 - Holds return address when called with Branch & Link instruction (B&L)
- PC - program counter (R15)



Program Counter (r15)

- When the processor is executing in ARM state:
 - All instructions are 32 bits wide
 - All instructions must be word aligned
 - Therefore the **pc** value is stored in bits [31:2] with bits [1:0] undefined (because ARM instructions cannot be halfword or byte aligned)
- **When the processor is executing in Thumb state:**
 - All instructions are 16 bits wide
 - All instructions must be halfword aligned
 - Therefore the **pc** value is stored in bits [31:1] with bit [0] undefined (because Thumb instructions cannot be byte aligned)

Special Registers

- Contain the processor status and define the operation states and interrupt/exception masking
 - development of simple applications do not require access to these registers
 - are needed for development of an embedded OS
- Special registers are not memory mapped, and can be accessed using special register access instructions such as **MSR** and **MRS**:

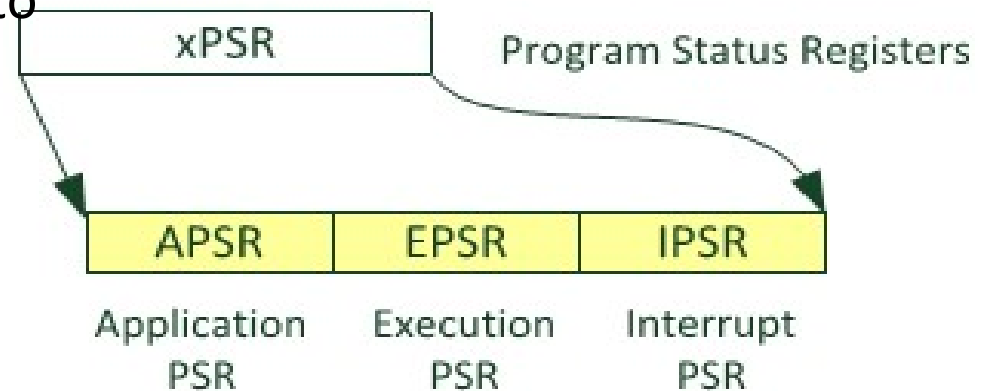
MRS <reg>, <special_reg>; Move special register into register

MSR <special_reg>, <reg>; Move register to special register

Program Status Registers

xPSR - combined Program Status Register (PSR) provides information about program execution and the ALU flags.

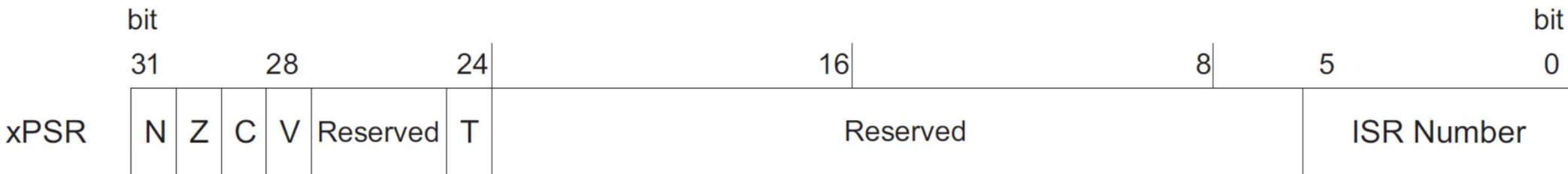
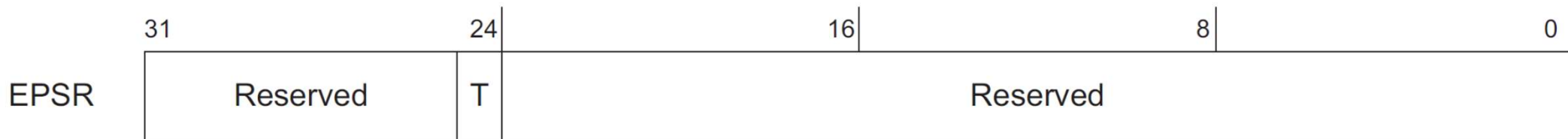
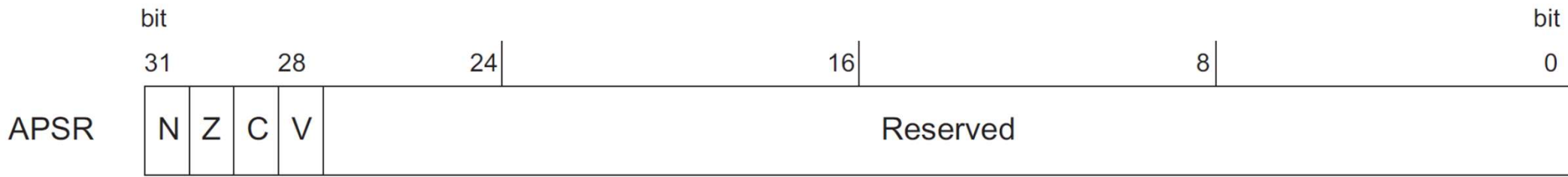
- The APSR contains the ALU flags to control conditional branches
- ERSR cannot be accessed by software code
- IPSR is read only, contains the current executing ISR (Interrupt Service Routine) number



	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												Exception Number				
EPSR						ICI/IT	T				ICI/IT					

Program Status Registers

Cortex-M0 and Cortex-M0+



Program Status Registers

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
xPSR	N	Z	C	V	Q	ICI/IT	T		GE*	ICI/IT		Exception Number				

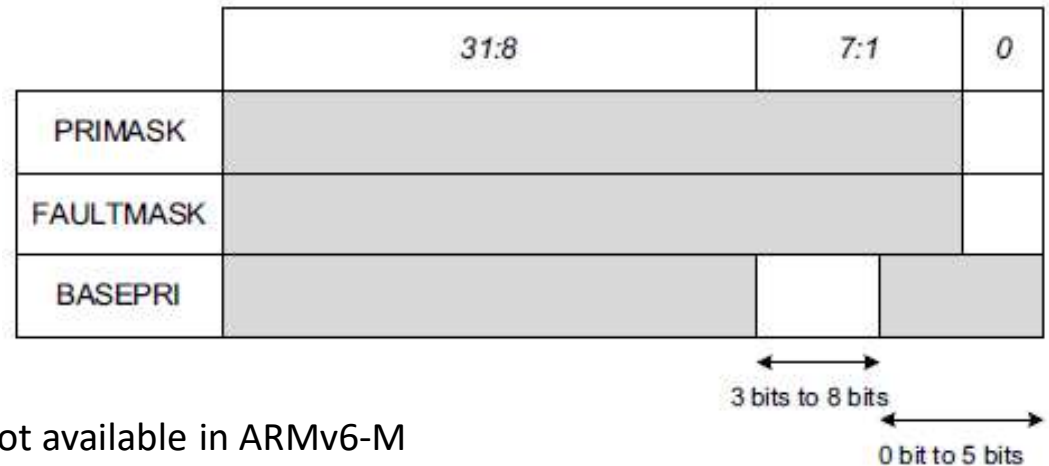
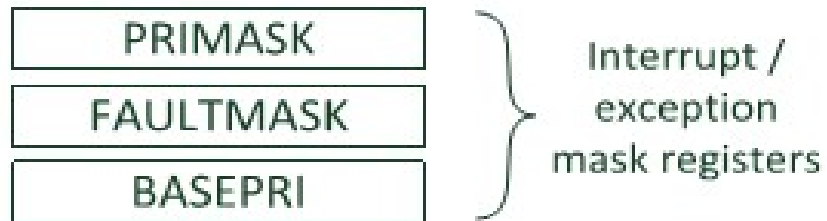
- Condition code flags
 - N = **N**egative result from ALU
 - Z = **Z**ero result from ALU
 - C = Result from ALU > 32 bits
 - V = Result from ALU > 31 bits
- Sticky Overflow flag - Q flag
 - Not available in ARMv6-M
 - Indicates if saturation has occurred
- ICI/IT bits
 - Interrupt-Continuable Instruction (ICI) bits, IF-THEN instruction status bit for conditional execution of Thumb2 instruction groups
- T Bit: Thumb state
 - Always T = 1; trying to clear this bit will cause a fault exception
- Exception Number
 - Indicates which exception the processor is handling
- New bits in ARMv7E-M
 - **GE[3:0]** used by some SIMD instructions

Program Status Registers

- **Compare** instructions **automatically update** the xPSR
- Most other instructions (e.g. arithmetic, logical, or shifting operations) **do not automatically update** the xPSR but can be forced to by adding the “**S**” directive after the

Other Special Registers

Cortex-M



Note: The FAULTMASK and BASEPRI registers are not available in ARMv6-M

- used for exception or interrupt masking
- Can only be accessed in the privileged access level
- When PRIMASK set, it blocks all exceptions and interrupts apart from the Non-Maskable Interrupt (NMI) and the HardFault exception
- When FAULTMASK set, it blocks all exceptions, interrupts and the HardFault exception
- BASEPRI masks exceptions or interrupts based on priority level. When it is set to a non-zero value, it blocks exceptions/interrupts that have the same or lower priority level.

Special Registers

- **CONTROL register defines:**
 - The selection of stack pointer (Main Stack Point/Process Stack Pointer)
 - Access level in Thread mode (Privileged/Unprivileged)
- CONTROL register can only be modified in the privileged access level

Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.

Stack

- Memory location is organized as a LIFO (Last In, First Out):
 - Stack pointer indicates the last stacked item on the stack memory.
- Two stacks, two independent stack pointers.
 - Handler mode always uses the MSP (Main Stack Pointer)
 - Thread mode can use MSP (Main Stack Pointer) by default, or PSP (Process Stack Pointer).
- In an OS environment, ARM recommends that threads running in Thread mode use the process stack and the kernel and exception handlers use the main stack
- **The stack can fill up quickly, depending on the situation**

Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM architecture**
 - ARM versions.
 - ARM programming model.
 - **ARM memory organization.**
 - ARM Instruction Set
 - ARM data processing operations.
 - ARM flow control operations.
- Summary



ARMv6 Endianness

- Instructions are always little-endian
- Loads and stores to Private Peripheral Bus are always little-endian
- Data: Depends on implementation, or from reset configuration
 - Kinetis processors are little-endian

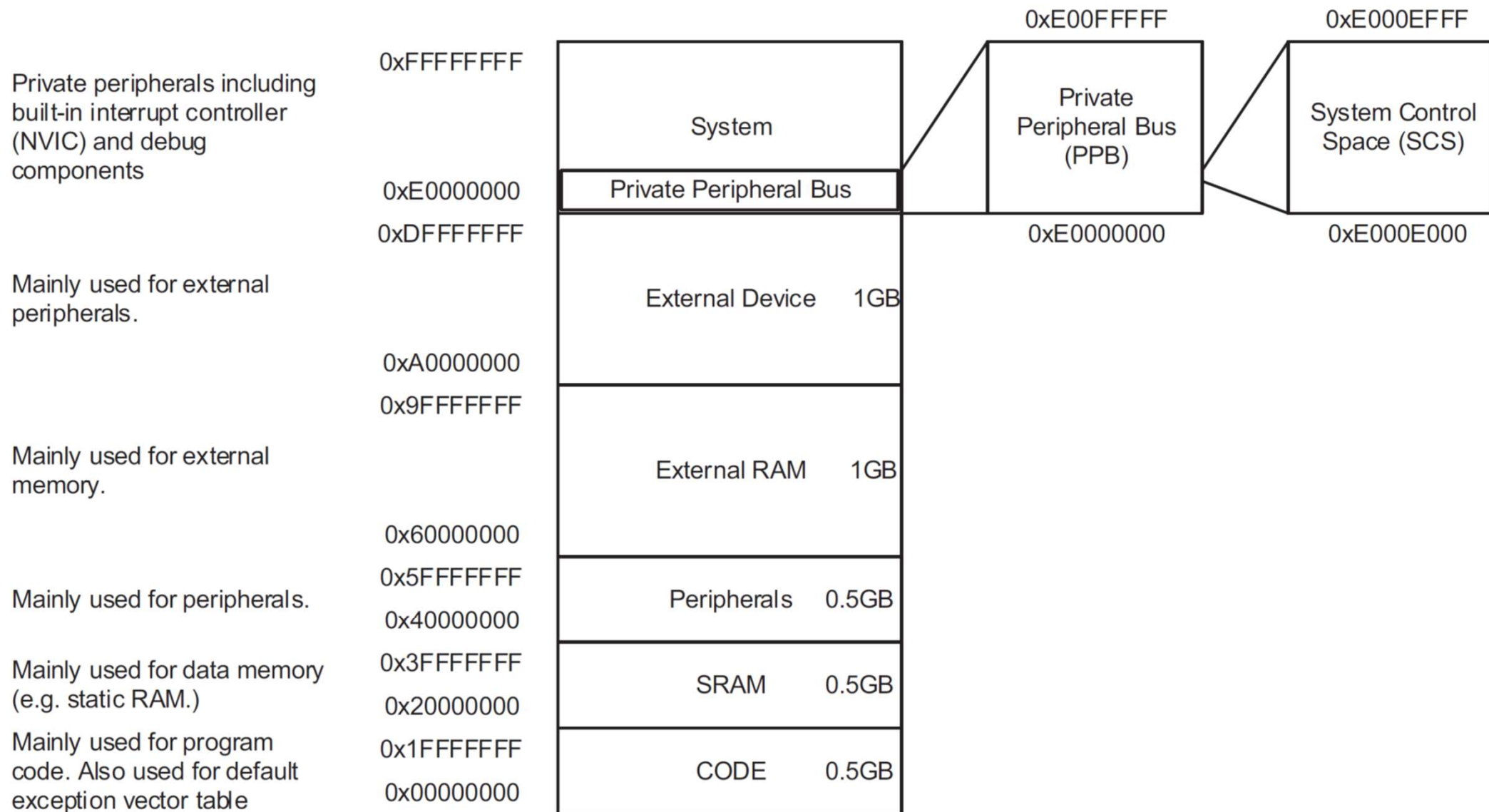
ARM memory organization

Memory model

- 32-bit address space:
 - Linear 4 GB address space removes complex paging schemes, simplifying software architecture
- Virtual memory is not supported in ARMv6-M.
- support fetching instructions from 16-bit flash memories
 - make the microcontroller design simpler, smaller, and consequently cheaper
- Data accesses are always naturally aligned

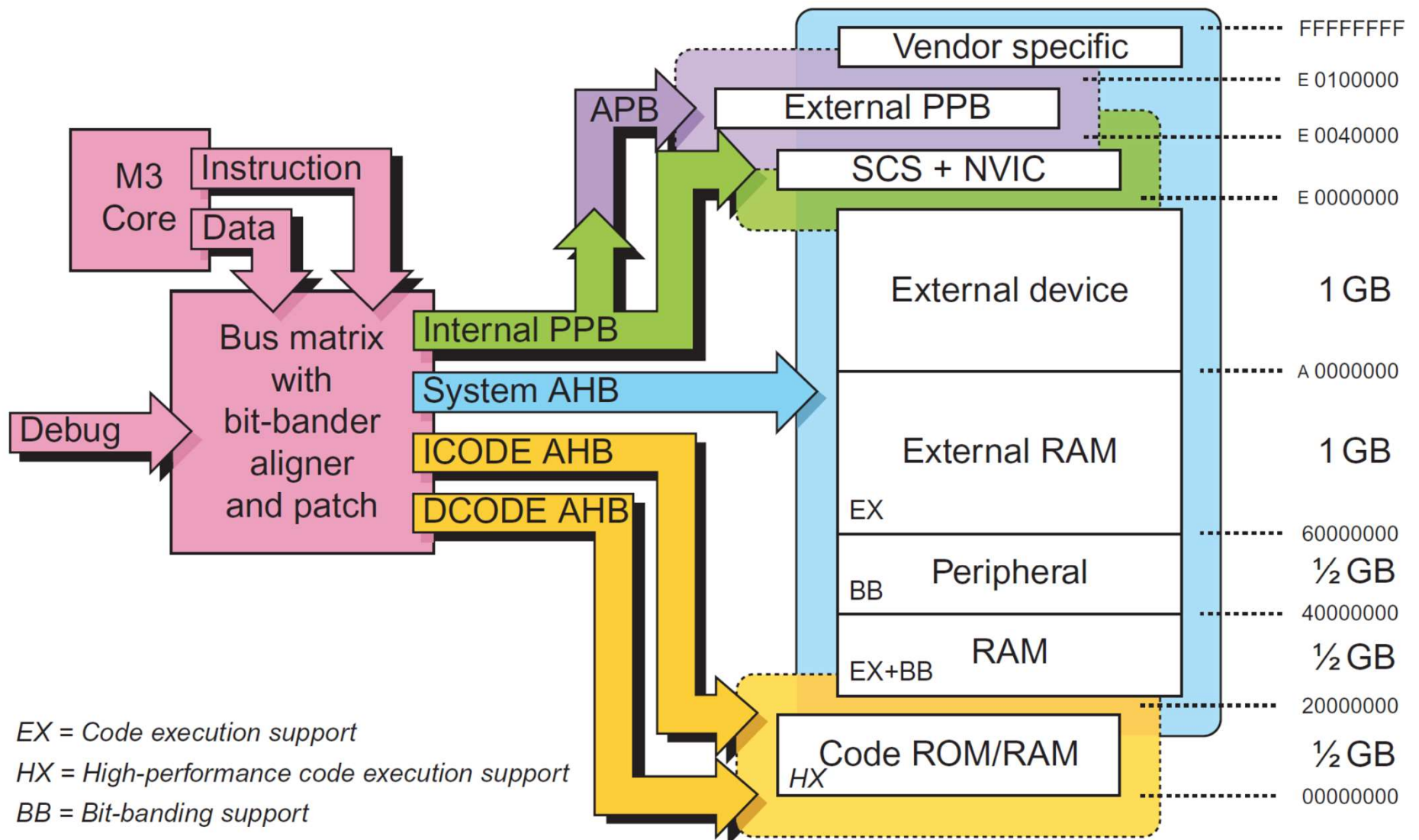
ARM memory organization

The Cortex-M memory map



ARM memory organization

The Cortex-M memory map and buses



Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM architecture**
 - ARM versions.
 - ARM programming model.
 - ARM memory organization.
 - **ARM Instruction Set**
 - ARM data processing operations.
 - ARM flow controlling operations.
- Summary



ARM and Thumb Instructions

- Thumb technology reduces program memory size and bandwidth requirements
- Thumb provides a **subset of ARM 32-bit instructions** re-encoded into usually fewer bits (most 16 bits, some 32 bits)
 - Not all 32-bit instructions available
 - Most 16-bit instructions can only access low registers (R0-R7), but a few can access high registers (R8-R15)
- 1995 :Thumb-1 instruction set
 - 16-bit instructions
- 2003: Thumb-2 instruction set
 - Adds some 32 bit instructions
 - Improves speed with little memory overhead
- CPU operating state
 - CPU decodes instructions based on whether in Thumb state or ARM state - controlled by T bit
 - Thumb state indicated by program counter being odd (LSB = 1)
- Cortex-M0+ only uses Thumb instructions, is always in Thumb state
- See ARMv6-M Architecture Reference Manual for specifics per instruction (Section A.6.7)

Cortex-M Instruction Groups

Group	Instr bits	Instructions	M0,M0+,M1	M3	M4	M7	M23	M33,M35P
Thumb-1	16	ADC, ADD, ADR, AND, ASR, B, BIC, BKPT, BLX, BX, CMN, CMP, CPS, EOR, LDM, LDR, LDRB, LDRH, LDRSB, LDRSH, LSL, LSR, MOV, MUL, MVN, NOP, ORR, POP, PUSH, REV, REV16, REVSH, ROR, RSB, SBC, SEV, STM, STR, STRB, STRH, SUB, SVC, SXTB, SXTB, TST, UXTB, UXTB, WFE, WFI, YIELD	Yes	Yes	Yes	Yes	Yes	Yes
Thumb-1	16	CBNZ, CBZ	No	Yes	Yes	Yes	Yes	Yes
Thumb-1	16	IT	No	Yes	Yes	Yes	No	Yes
Thumb-2	32	BL, DMB, DSB, ISB, MRS, MSR	Yes	Yes	Yes	Yes	Yes	Yes
Thumb-2	32	SDIV, UDIV	No	Yes	Yes	Yes	Yes	Yes
Thumb-2	32	ADC, ADD, ADR, AND, ASR, B, BFC, BFI, BIC, CDP, CLREX, CLZ , CMN, CMP, DBG, EOR, LDC, LDM, LDR, LDRB, LDRBT, LDRD, LDREX, LDREXB, LDREXH, LDRH, LDRHT, LDRSB, LDRSBT, LDRSH, LDRSHT, LDRT, LSL, LSR, MCR, MCRR, MLA, MLS, MOV, MOVT, MRC, MRRC, MUL, MVN, NOP, ORN, ORR, PLD, PLDW, PLI, POP, PUSH, RBIT, REV, REV16, REVSH, ROR, RRX, RSB, SBC, SBFX, SEV, SMLAL, SMULL, SSAT, STC, STM, STR, STRB, STRBT, STRD, STREX, STREXB, STREXH, STRH, STRHT, STRT, SUB, SXTB, SXTB, TBB, TBH, TEQ, TST, UBFX, UMLAL, UMULL, USAT, UXTB, UXTB, WFE, WFI, YIELD	No	Yes	Yes	Yes	No	Yes
DSP	32	PKH, QADD, QADD16, QADD8, QASX, QDADD, QDSUB, QSAX, QSUB, QSUB16, QSUB8, SADD16, SADD8, SASX, SEL, SHADD16, SHADD8, SHASX, SHSAX, SHSUB16, SHSUB8, SMLABB, SMLABT, SMLATB, SMLATT, SMLAD, SMLALBB, SMLALBT, SMLALTB, SMLALTT, SMLALD, SMLAWB, SMLAWT, SMLSD, SMLSLD, SMMLA, SMMLS, SMMUL, SMUAD, SMULBB, SMULBT, SMULTT, SMULTB, SMULWT, SMULWB, SMUSD, SSAT16, SSAX, SSUB16, SSUB8, SXTAB, SXTAB16, SXTAH, SXTB16, UADD16, UADD8, UASX, UHADD16, UHADD8, UHASX, UHSAX, UHSUB16, UHSUB8, UMAAL, UQADD16, UQADD8, UQASX, UQSAX, UQSUB16, UQSUB8, USAD8, USADA8, USAT16, USAX, USUB16, USUB8, UXTAB, UXTAB16, UXTAH, UXTB16	No	No	Yes	Yes	No	Optional
SP Float	32	VABS, VADD, VCMPE, VCMPE, VCVT, VCVTR, VDIV, VLDM, VLDR, VMLA, VMLS, VMOV, VMRS, VMSR, VMUL, VNEG, VNMLA, VNMLS, VNMUL, VPOP, VPOP, VSQRT, VSTM, VSTR, VSUB	No	No	Optional	Optional	No	Optional
DP Float	32	VCVTA, VCVTM, VCVTN, VCVTP, VMAXNM, VMINNM, VRINTA, VRINTM, VRINTN, VRINTP, VRINTR, VRINTX, VRINTZ, VSEL	No	No	No	Optional	No	No
TrustZone	16	BLXNS, BXNS	No	No	No	No	Optional	Optional
TrustZone	32	SG, TT, TTT, TTA, TTAT	No	No	No	No	Optional	Optional
Co-processor	16	CDP, CDP2, MCR, MCR2, MCRR, MCRR2, MRC, MRC2, MRRC, MRRC2	No	No	No	No	No	Optional

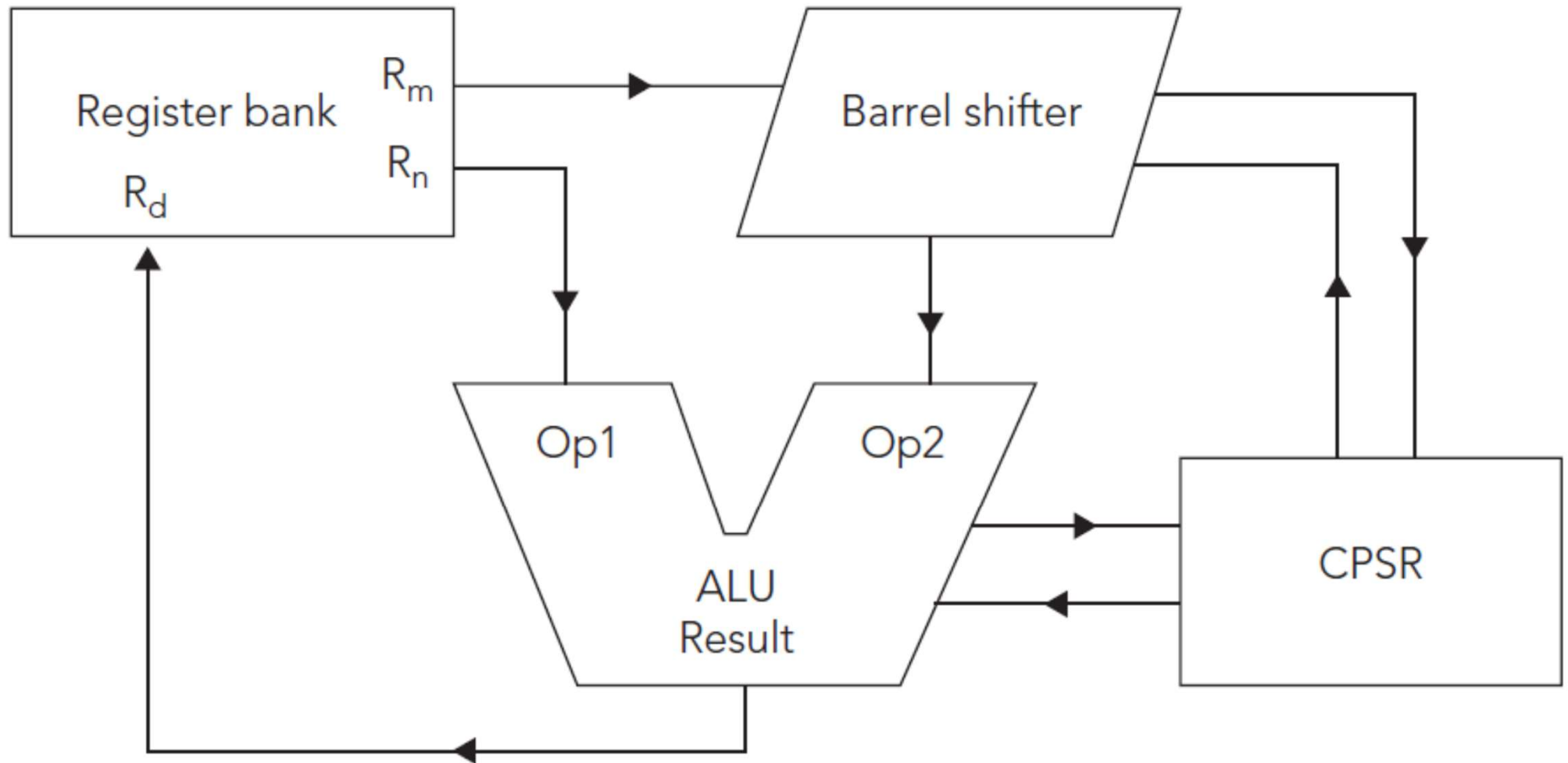
https://en.wikipedia.org/wiki/ARM_Cortex-M#Instruction_sets

Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM architecture**
 - ARM versions.
 - ARM programming model.
 - ARM memory organization.
 - ARM Instruction Set
 - **ARM data processing operations.**
 - Conditional Codes
 - Load/store instructions
 - ARM flow controlling operations.
- Summary



Data processing instructions



ARM Calculation Unit

Data processing Instructions

- The data processing instructions only work on registers, **NOT memory**.

- Instruction Format:

<Operation>{<cond>}{S} Rd, Rn, Operand2

- **<Operation>**: 3-letter mnemonic contains code of operation
 - ADD, SUB, ...
- **{<cond>}**: Optional two-letter condition code
 - ADDE**Q** r0, r1, r2
- **{S}**: Optional additional flag
 - ADD**S** r0, r1, r2
- **Rd**: Destination register
 - ADD **r0**, r1, r2
- **Rn**: First operand register
 - ADD r0, **r1**, r2

Data processing Instructions

- The data processing instructions only work on registers, **NOT memory**.

- Instruction Format:

<Operation>{<cond>}{S} Rd, Rn, Operand2

- Operand 2 can be a register or an (8-bit) immediate value

- **SUB r0, r1, r2** ($r0 = r1 - r2$)

- **ADD r1, r4, #0xFF** ($r1 = r4 + 0xFF$)

- Comparisons set flags only - they do not specify Rd

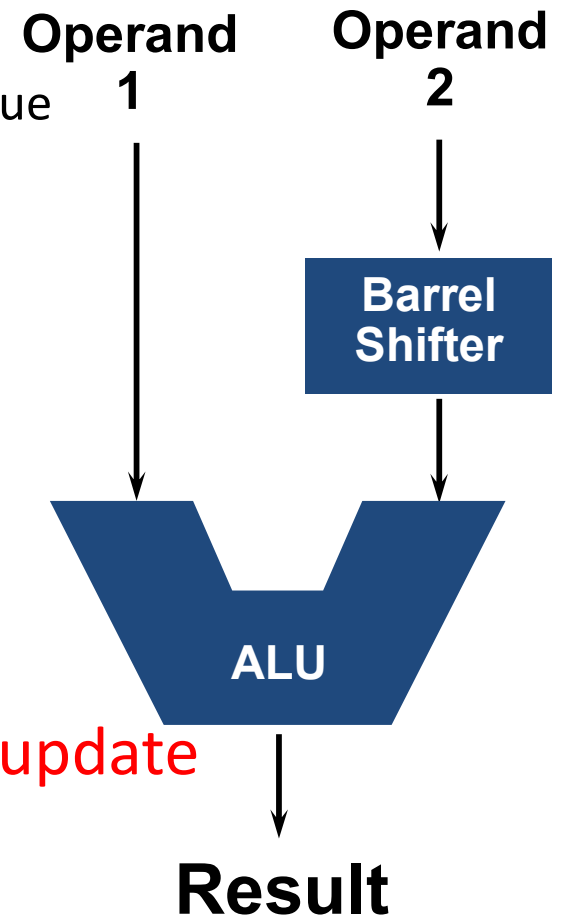
- **CMP r0, r3**

- Data movement does not specify Rn

- **MOV r0, r1**

- Operand 2 is sent to the ALU via barrel shifter

- **By default, data processing instructions do not update the condition flags.**



Data processing Instructions

Arithmetic:

- ADD, ADC : add (w. carry)
- SUB, SBC : subtract (w. carry)
- RSB, RSC : reverse subtract (w. carry)

Logical:

- AND:
- ORR:
- EOR: Exclusive OR
- BIC : bit clear
 - BIC r0,r1,r2 ; r0 = r1 and (not r2).

Data processing Instructions

ARM comparison instructions

- CMP : compare (using subtraction on operands)
- CMN : negated compare (using addition on operands)
- TST : bit-wise test (using AND on operands)
- TEQ : bit-wise negated test (using EOR on operands)

These instructions set only the NZCV bits of xPSR.

Data processing Instructions

ARM move instructions

- MOV, MVN : move (negated: one's complement of operand bits)
 - put values only from registers or from immediate values into registers

MOV r0, r1 ; sets r0 to r1

Mov r0, #4

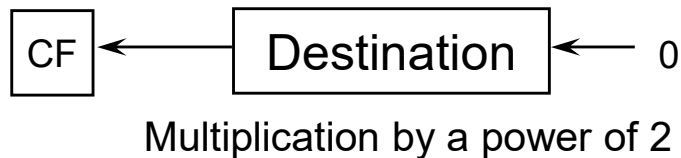
Mov r0, r1, lsr #1

Mov r0, r1, lsr r2

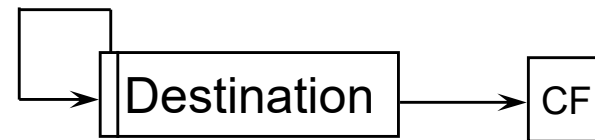
ARM data instructions

- **Shift operations** are used as part of data processing instructions.
 - bits can be shifted from 0-31 places, typically without performance penalty

LSL: Logical Shift Left



ASR: Arithmetic Shift Right



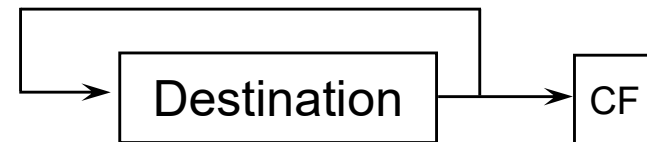
Division by a power of 2, preserving the sign bit

LSR: Logical Shift Right



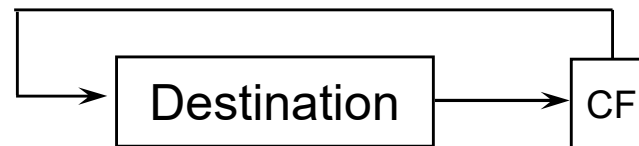
Division by a power of 2

ROR: Rotate Right



Bit rotate with wrap around from LSB to MSB

RRX: Rotate Right Extended



Single bit rotate with wrap around from CF to MSB

Shift Operations



- ARM cores in ARM mode **do not actually need shift instructions, the barrel shifter can perform a shift during an instruction by specifying a shift on the second operator directly inside an instruction.**
- In Thumb mode, the instructions are simplified and shift instructions do exist.

ADD r0, r5, r5, LSL #1 ; $r0 = r5 \times 3$

MOVS R1,R0,LSR #3 ; $R1 = R0 \gg 3$

Multiply and Divide

- There are 2 classes of multiply - producing 32-bit and 64-bit results
- 32-bit versions on an ARM7TDMI will execute in 2 - 5 cycles

- `MUL r0, r1, r2` ; `r0 = r1 * r2`
 - `MLA r0, r1, r2, r3` ; `r0 = (r1 * r2) + r3`

- 64-bit multiply instructions offer both signed and unsigned versions
 - For these instruction there are 2 destination registers

- `[U|S]MULL r4, r5, r2, r3` ; `r5:r4 = r2 * r3`
 - `[U|S]MLAL r4, r5, r2, r3` ; `r5:r4 = (r2 * r3) + r5:r4`

- Most ARM cores do not offer integer divide instructions
 - Division operations will be performed by C library routines or inline shifts

Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM instruction set**
 - ARM versions.
 - ARM programming model.
 - ARM memory organization.
 - ARM data processing operations.
 - **Coditional Codes**
 - Load/store instructions
 - ARM flow of control.
- Summary



Condition Codes

- All operations can be performed conditionally by testing N, Z, C, and V flags in xPSR:
 - Note AL is the default and does not need to be specified

Suffix	Description	Flags tested
EQ	Equal	Z=1
NE	Not equal	Z=0
CS/HS	Unsigned higher or same	C=1
CC/LO	Unsigned lower	C=0
MI	Minus	N=1
PL	Positive or Zero	N=0
VS	Overflow	V=1
VC	No overflow	V=0
HI	Unsigned higher	C=1 & Z=0
LS	Unsigned lower or same	C=0 or Z=1
GE	Greater or equal	N=V
LT	Less than	N!=V
GT	Greater than	Z=0 & N=V
LE	Less than or equal	Z=1 or N!=V
AL	Always	

Conditional Execution and Flags

- ARM instructions can be made to execute conditionally by post-fixing them with the appropriate condition code
 - This can increase code density and increase performance by reducing the number of forward branches

<code>CMP</code>	<code>r0, r1</code>	←	<code>r0 - r1, compare r0 with r1 and set flags</code>
<code>ADDGT</code>	<code>r2, r2, #1</code>	←	<code>if > then r2=r2+1 flags remain unchanged</code>
<code>ADDE</code>	<code>r3, r3, #1</code>	←	<code>if <= then r3=r3+1 flags remain unchanged</code>

- By default, data processing instructions do not affect the condition flags but this can be achieved by post fixing the instruction (and any condition code) with an "S"

`loop`

<code>ADD</code>	<code>r2, r2, r3</code>	←	<code>r2=r2+r3</code>
<code>SUBS</code>	<code>r1, r1, #0x01</code>	←	<code>decrement r1 and set flags</code>
<code>BNE</code>	<code>loop</code>	←	<code>if Z flag clear then branch</code>

Conditional execution examples

C source code

```
if (r0 == 0)
{
    r1 = r1 + 1;
}
else
{
    r2 = r2 + 1;
}
```

ARM instructions

unconditional

```
CMP r0, #0
BNE else
ADD r1, r1, #1
B end
else
    ADD r2, r2, #1
end
...
```

conditional

```
CMP r0, #0
ADDEQ r1, r1, #1
ADDNE r2, r2, #1
...
```

- 5 instructions
- 5 words
- 5 or 6 cycles

- 3 instructions
- 3 words
- 3 cycles

Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM instruction set**
 - ARM versions.
 - ARM programming model.
 - ARM memory organization.
 - ARM data processing operations.
 - Condition Codes
 - **Load/store memory access instructions**
 - ARM flow controlling operations.
- Summary



ARM load/store instructions

LDR	STR	Word
LDRB	STRB	Byte
LDRH	STRH	Halfword
LDRSB		Signed byte load
LDRSH		Signed halfword load

- Memory system must support all access sizes
- Syntax:
 - **LDR**{<cond>} {<size>} Rd, <address>
 - **STR**{<cond>} {<size>} Rd, <address>

ARM load/store instructions

- Addressing modes: Address accessed by LDR/STR is specified by a base register with an offset
 - register indirect : `LDR r0, [r1]`
 - loads r0 from the address given by r1
 - with second register : `LDR r0, [r1, -r2]`
 - loads r0 from the address given by $r1 - r2$
 - with constant : `LDR r0, [r1, #4]`
 - loads r0 from the address $r1 + 4$.
 - Auto-indexing increments base register: `LDR r0, [r1, #4]!`
 - $R1 = r1 + 4$ and loads r0 from the address r1
 - Post-indexing fetches, then does offset: `LDR r0, [r1], #4`
 - Loads r0 from r1, then adds 4 to r1.

Loading 32 bit constants

- To allow larger constants to be loaded, the assembler offers a pseudo-instruction:
 - **LDR rd, =const**
- This will either:
 - Produce a **MOV** or **MVN** instruction to generate the value (if possible)or
 - Generate a **LDR** instruction with a PC-relative address to read the constant from a *literal pool* (Constant data area embedded in the code)
- For example
 - **LDR r0, =0xFF** => **MOV r0, #0xFF**
 - **LDR r0, =0x55555555** => **LDR r0, [PC, #Imm12]**
 ...
 ...
 DCD 0x55555555
- This is the recommended way of loading constants into a register

Example: C assignments

- C:

$x = (a + b) - c;$

- Assembly:

```
LDR r4,=a           ; get address for a
LDR r0,[r4]          ; get value of a
LDR r4,=b           ; get address for b, reusing r4
LDR r1,[r4]          ; get value of b
ADD r3,r0,r1         ; compute a+b
LDR r4,=c           ; get address for c
LDR r2,[r4]          ; get value of c
```

C assignment, cont'd.

```
SUB  r3,r3,r2      ; complete computation of x
LDR  r4,=x         ; get address for x
STR  r3, [r4]      ; store value of x
```

Example: C assignment

- C:

`y = a*(b+c);`

- Assembly:

`LDR r4,=b ; get address for b`

`LDR r0,[r4] ; get value of b`

`LDR r4,=c ; get address for c`

`LDR r1,[r4] ; get value of c`

`ADD r2,r0,r1 ; compute partial result`

`LDR r4,=a ; get address for a`

`LDR r0,[r4] ; get value of a`

C assignment, cont'd.

```
MUL r2,r2,r0 ; compute final value for y  
LDR r4,=y ; get address for y  
STR r2,[r4] ; store y
```

Example: C assignment

- C:

```
z = (a << 2) | (b & 15);
```

- Assembler:

```
LDR r4,=a ; get address for a
LDR r0,[r4] ; get value of a
MOV r0,r0, LSL 2 ; perform shift
LDR r4,=b ; get address for b
LDR r1,[r4] ; get value of b
AND r1,r1,#15 ; perform AND
ORR r1,r0,r1 ; perform OR
```

C assignment, cont'd.

```
LDR r4,=z ; get address for z  
STR r1,[r4] ; store value for z
```

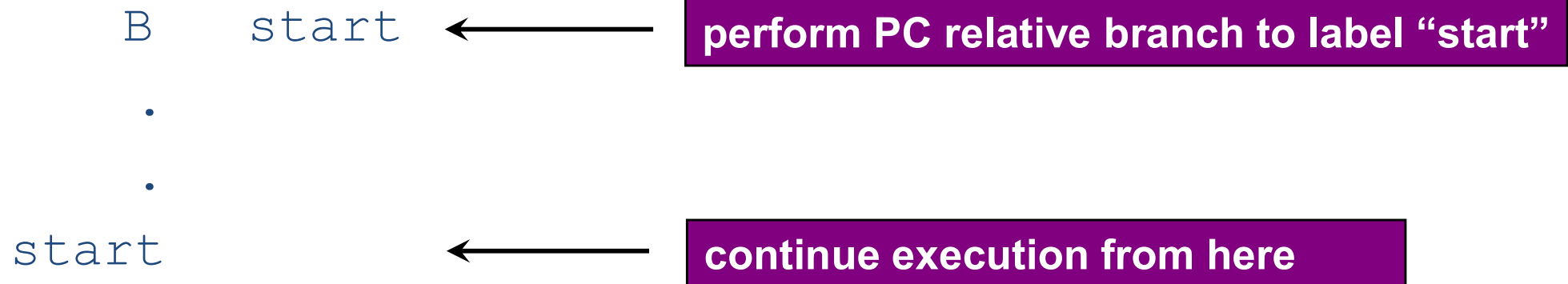
Outline

- Computer architecture taxonomy
- Basic concept of Instruction set
- **ARM instruction set**
 - ARM versions.
 - ARM programming model.
 - ARM memory organization.
 - ARM data operations.
 - Codition Codes
 - Load/store instructions
 - **ARM flow controlling operations.**
- Summary



Branch Instructions

- Branch instructions have the following format:
 - **B**{**L**} {<cond>} label
 - subroutine calls can be made by specifying the optional {**L**}
 - a 24 bit address offset field is part of the instruction encoding
 - On execution this is left shifted 2 places (since ARM instructions are always word aligned) to give a 26 bit value, thus giving a relative branch range of ± 32 MB
 - Causes a pipeline flush



Example: if statement

- C:
`if (a < b) { x = 5; y = c + d; } else x = c - d;`
- Assembly:
; compute and test condition
LDR r4,=a ; get address for a
LDR r0,[r4] ; get value of a
LDR r4,=b ; get address for b
LDR r1,[r4] ; get value for b
CMP r0,r1 ; compare a < b
BGE fblock ; if a >= b, branch to false block

If statement, cont'd.

```
; true block: unconditional
MOV r0,#5 ; generate value for x
LDR r4,=x ; get address for x
STR r0,[r4] ; store x
LDR r4,=c ; get address for c
LDR r0,[r4] ; get value of c
LDR r4,=d ; get address for d
LDR r1,[r4] ; get value of d
ADD r0,r0,r1 ; compute y
LDR r4,=y ; get address for y
STR r0,[r4] ; store y
B after ; branch around false block
```

If statement, cont'd.

; false block

fblock LDR r4,=c ; get address for c

LDR r0,[r4] ; get value of c

LDR r4,=d ; get address for d

LDR r1,[r4] ; get value for d

SUB r0,r0,r1 ; compute c-d

LDR r4,=x ; get address for x

STR r0,[r4] ; store value of x

after ...

Example: Conditional instruction implementation

```
; compute and test condition
LDR r4,=a ; get address for a
LDR r0,[r4] ; get value of a
LDR r4,=b ; get address for b
LDR r1,[r4] ; get value for b
CMP r0,r1 ; compare a < b
```

Example: Conditional instruction implementation

```
; true block: conditional
    MOVLT r0,#5 ; generate value for x
    LDRLT r4,x ; get address for x
    STRLT r0,[r4] ; store x
    LDRLT r4,c ; get address for c
    LDRLT r0,[r4] ; get value of c
    LDRLT r4,d ; get address for d
    LDRLT r1,[r4] ; get value of d
    ADDLT r0,r0,r1 ; compute y
    LDRLT r4,y ; get address for y
    STRLT r0,[r4] ; store y
```

Example: Conditional instruction implementation

```
; false block
LDRGE r4,=c ; get address for c
LDRGE r0,[r4] ; get value of c
LDRGE r4,=d ; get address for d
LDRGE r1,[r4] ; get value for d
SUBGE r0,r0,r1 ; compute c-d
LDRGE r4,=x ; get address for x
STRGE r0,[r4] ; store value of x
```

Example: switch statement

- C:

```
switch (test) { case 0: ... break; case 1: ... }
```

- Assembly:

```
LDR r2,=test ; get address for test
```

```
LDR r0,[r2] ; load value for test
```

```
LDR r1,=switchtab ; load address for switch table
```

```
LDR r15,[r1,r0,LSL #2] ; index switch table
```

```
switchtab DCD case0
```

```
          DCD case1
```

```
...
```

```
case0
```

```
    ADD r0, r1, r2 ; Operation 0
```

```
    B Endcase ; Break
```

```
case1
```

```
    SUB r0, r1,
```

```
Endcase
```

```
...
```


Example: FIR filter (1)

- C:

```
for (i=0, f=0; i<N; i++)  
    f = f + c[i]*x[i];
```

- Assembly

; loop initiation code

MOV r0,#0 ; use r0 for I

MOV r8,#0 ; use separate index for arrays

LDR r2,=N ; get address for N

LDR r1,[r2] ; get value of N

MOV r2,#0 ; use r2 for f

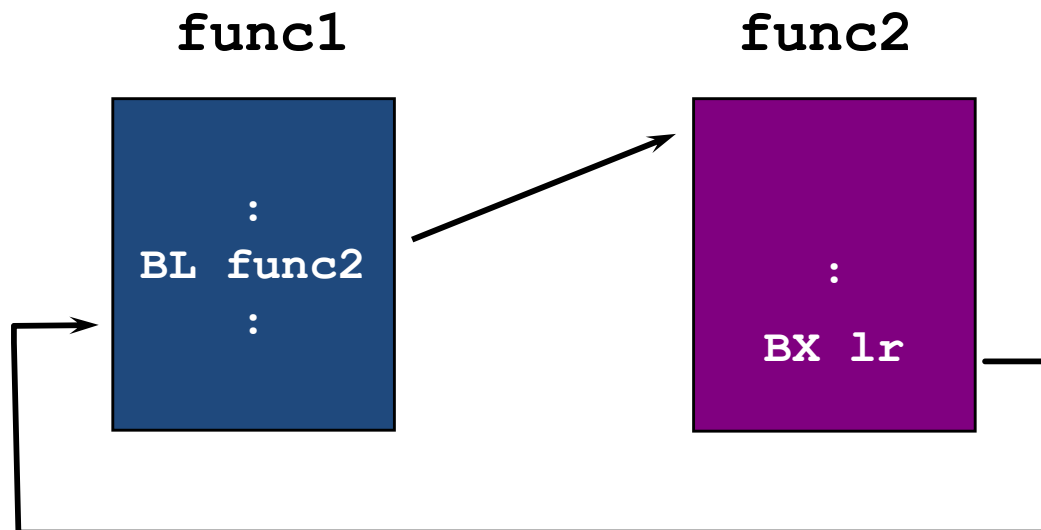
Example: FIR filter (2)

```
LDR r3,=c ; load r3 with base of c
LDR r5,=x ; load r5 with base of x
; loop body
loop LDR r4,[r3,r8] ; get c[i]
    LDR r6,[r5,r8] ; get x[i]
    MUL r4,r4,r6 ; compute c[i]*x[i]
    ADD r2,r2,r4 ; add into running sum
    ADD r8,r8,#4 ; add one word offset to array index
    ADD r0,r0,#1 ; add 1 to i
    CMP r0,r1 ; exit?
    BLT loop ; if i < N, continue
```

Subroutines

- Implementing a conventional subroutine call requires two steps:
 - Store the return address
 - Branch to the address of the required subroutine
- These steps are carried out in one instruction, **BL**
 - The return address is stored in the link register (**lr/r14**)
 - Branch to an address anywhere within a +/- 32MB range
- Returning is performed by restoring the program counter (**pc**) from **lr**
 - **MOV r15, r14**
 - or **BX lr**

```
void func1 (void)
{
    :
    func2 ();
    :
}
```



Generating Branches with LDR

- The ARM's branch instruction is limited to a range of $\pm 32\text{MB}$
 - However branches can also be performed by loading address values directly into the PC (**r15**)
 - **armasm** provides pseudo instructions to make this easier

Assembler Code

```
LDR pc, =label ; load address of label into PC
```

ARMASM

Branches anywhere within the 4GB address space are thus possible

Object Code

```
LDR pc, [pc, #n]
```

.

```
-----  
DCD 0x12345678
```

Literal pool address data



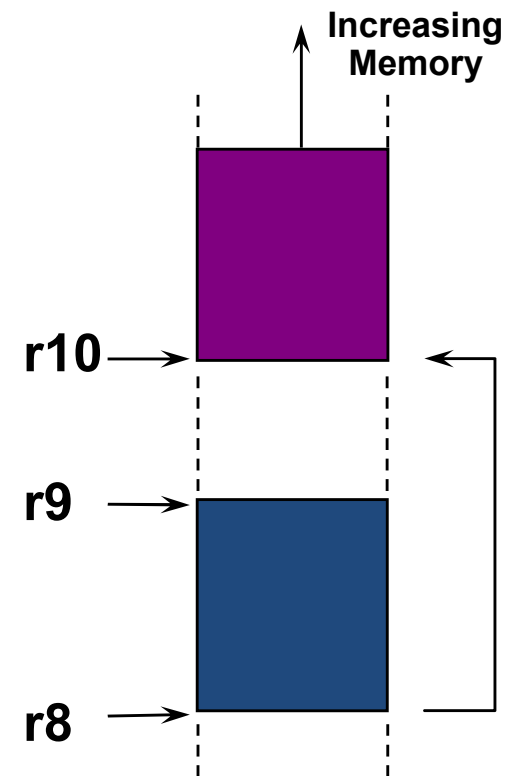
Memory Block Copying (1)

- The use of base register updating enables simple copying routines to be written
 - For example: The post-indexed variant could be used to copy a block of memory

```
; r8 points to start of source data  
; r9 points to end of source data  
; r10 points to start of destination data
```

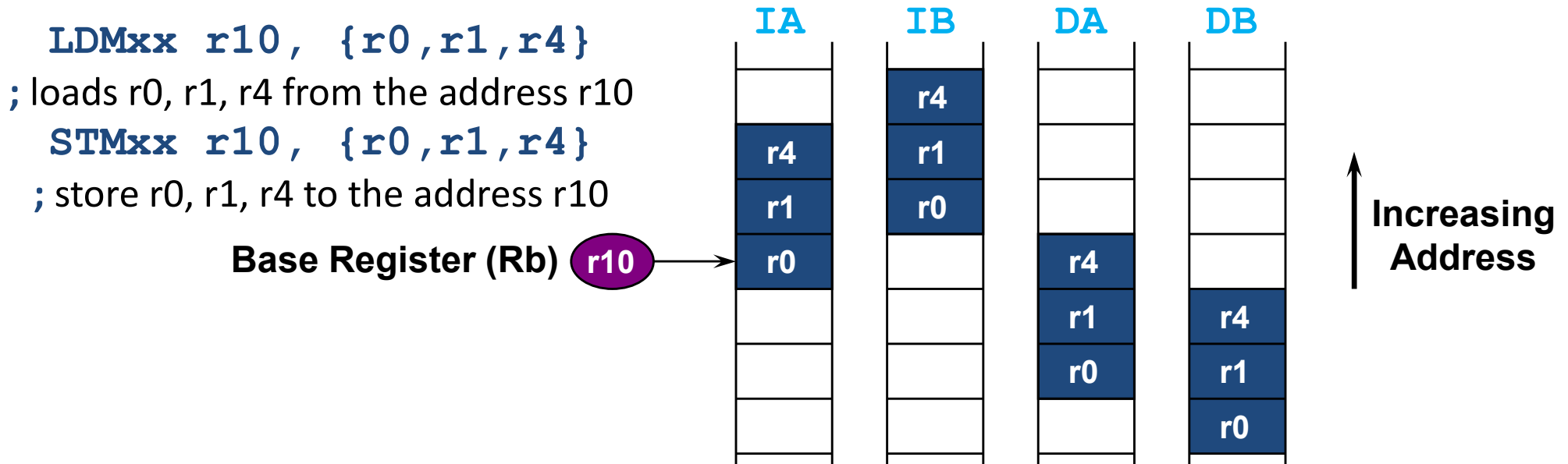
```
loop  LDR    r0, [r8], #4 ; load 4 bytes  
      STR    r0, [r10], #4; and store them  
      CMP    r8, r9      ; check for the end  
      BLT    loop        ; else loop
```

- In this example 1 word is copied per iteration



Load and Store Multiples

- Syntax:
 - **<LDM|STM>**{<cond>}<addressing_mode> Rb{!}, <register list>
- 4 addressing modes:
 - **LDMIA / STMIA** increment after
 - **LDMIB / STMIB** increment before
 - **LDMDA / STMDA** decrement after
 - **LDMDB / STMDB** decrement before



Memory Block Copying (2)

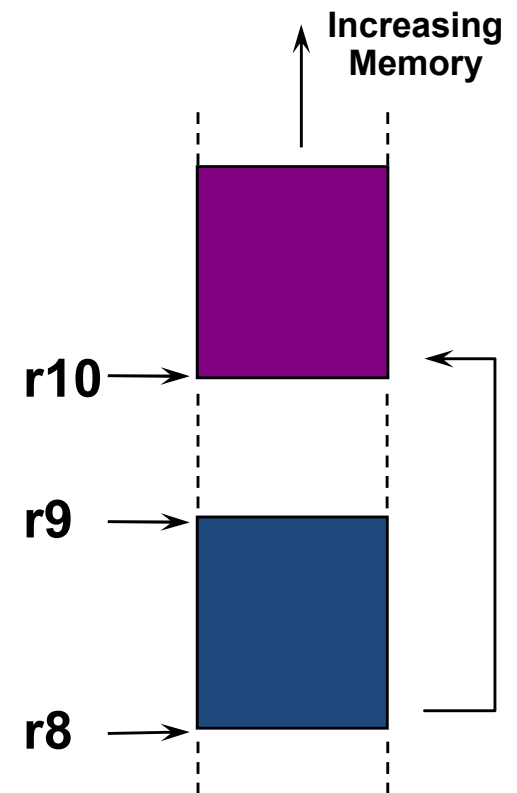
- As well as being used for stack operations, the `STM` / `LDM` instructions can perform block copying of memory

- For example

```
; r8 points to start of source data  
; r9 points to end of source data  
; r10 points to start of destination data
```

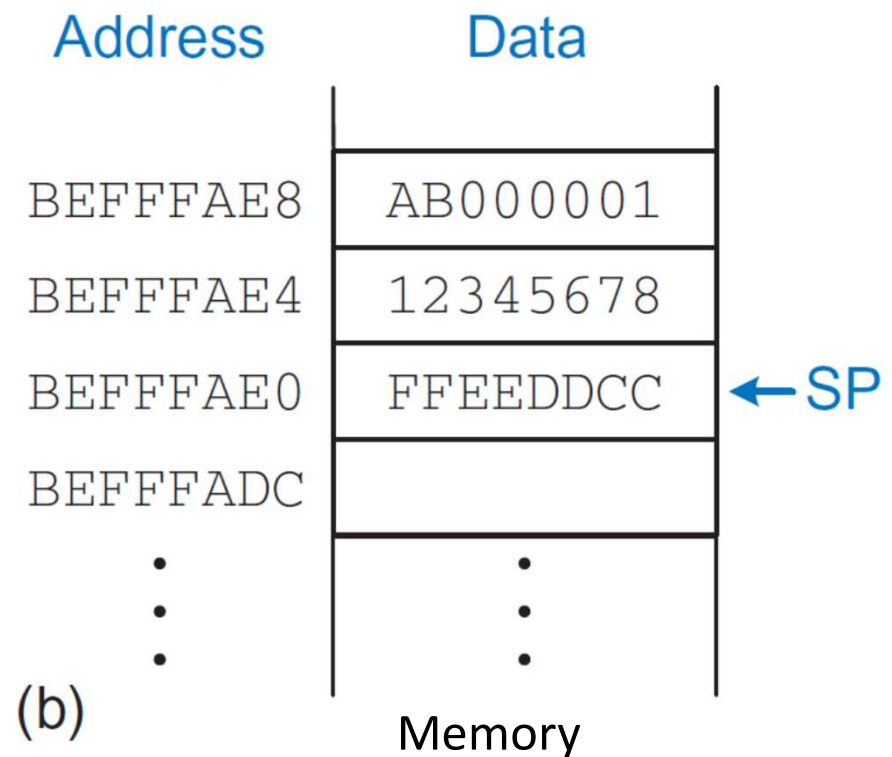
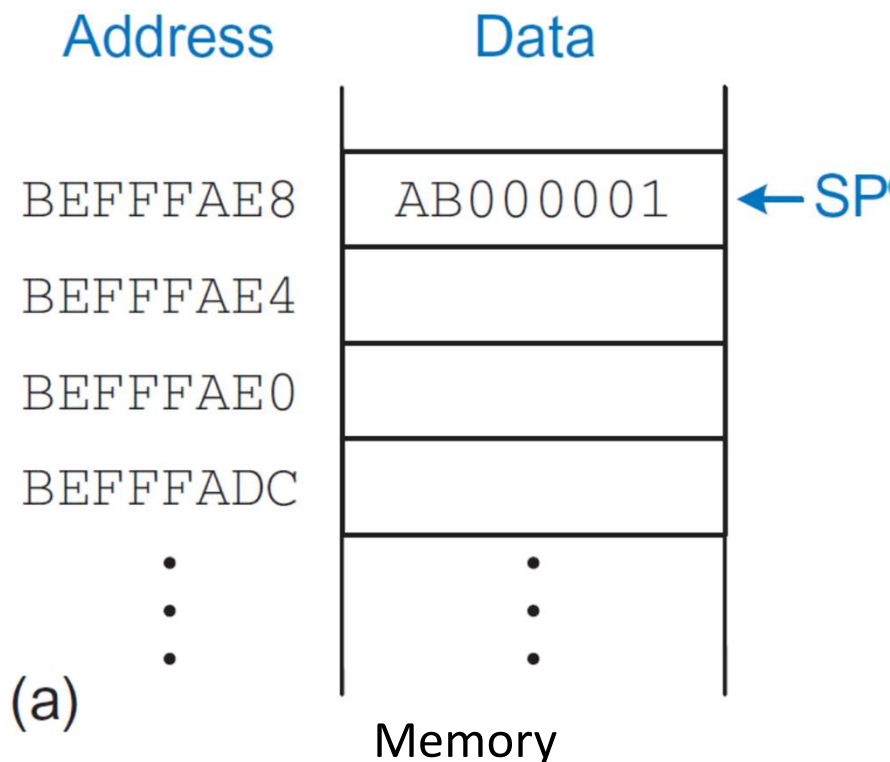
```
loop  LDMIA  r8!, {r0-r7} ; load 32 bytes  
      STMIA  r10!, {r0-r7}; and store them  
      CMP    r8, r9      ; check for the end  
      BLT    loop        ; and loop
```

- In this example 8 words are copied per loop



Stacks

- The stack is a last-in-first-out (LIFO) queue:
 - The top of the stack is the most recently allocated space
- ARM stack grows down to lower addresses in memory
 - The stack pointer, SP (R13), points to the top of the stack



The stack (a) before expansion and (b) after two-word expansion

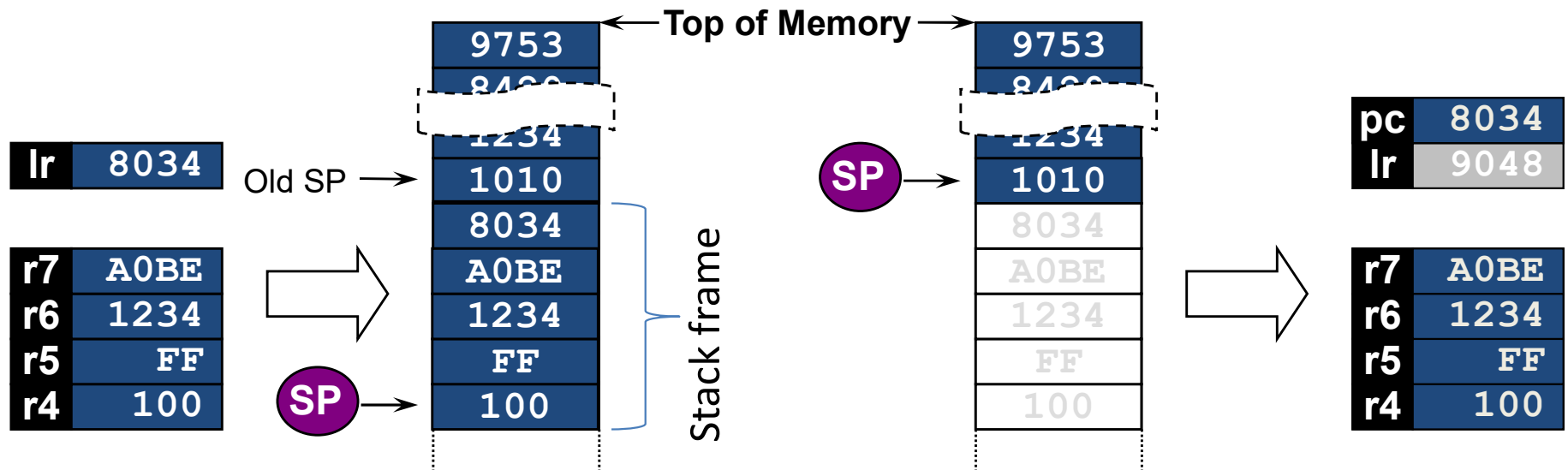
Stacks

- ARM stack operations are implemented by block transfer instructions:

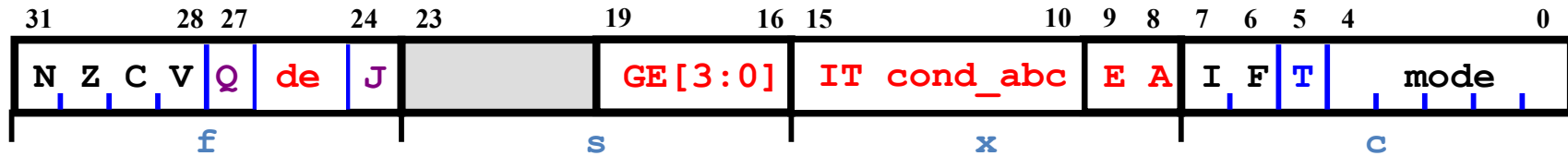
- **STMFD** (Push) Store Multiple - Full Descending stack [STMDB]
- **LDMFD** (Pop) Load Multiple - Full Descending stack [LDMIA]
 - Note: Multiple registers will always be stacked in register order from lowest register to lowest memory location
 - The order registers are specified has no effect.

STMFD *sp!*, {r4-r7, lr}

LDMFD *sp!*, {r4-r7, pc}



xPSR access

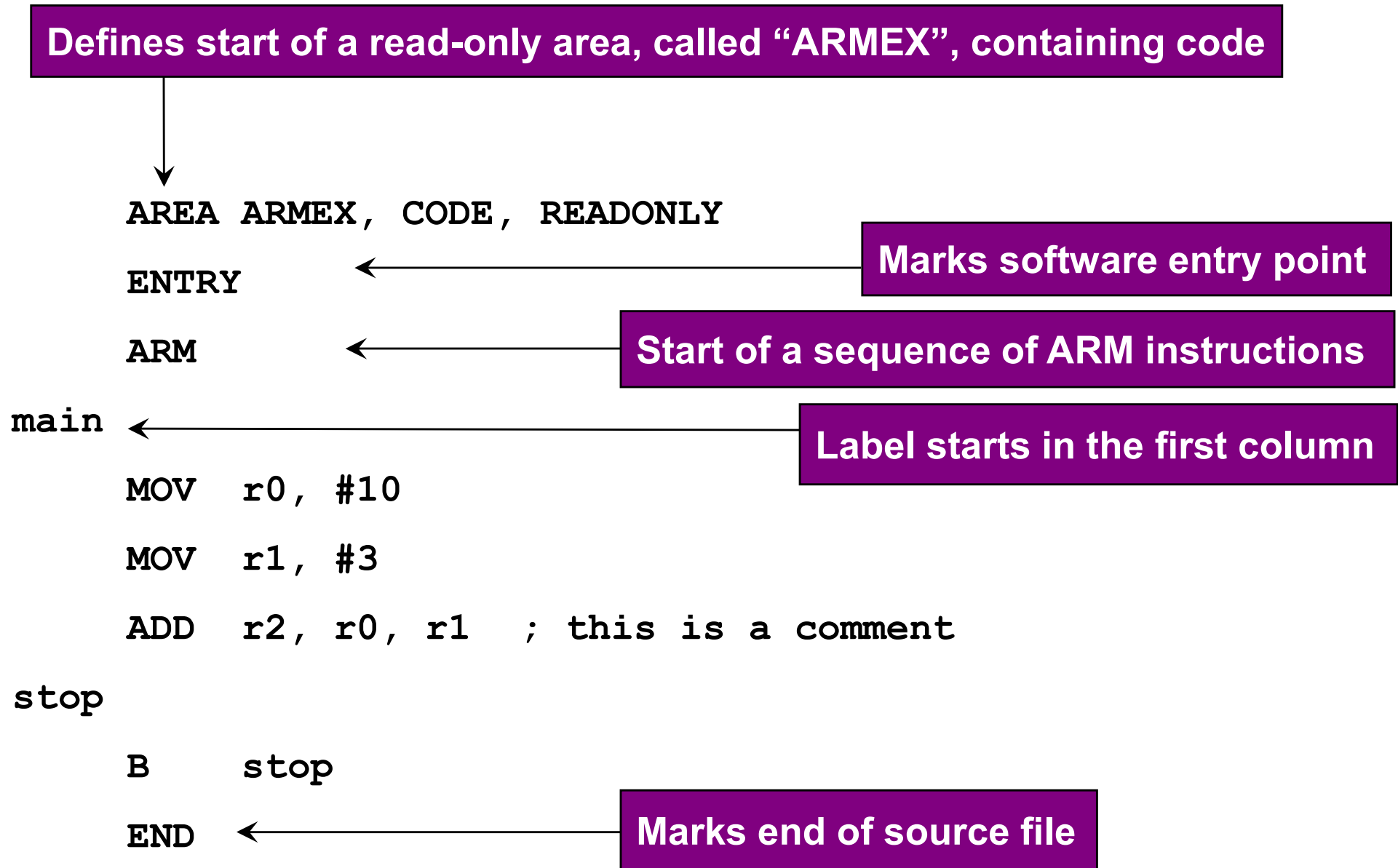


- MRS and MSR allow contents of xPSR to be transferred to / from a general purpose register or take an immediate value
 - MSR allows the whole status register, or just parts of it to be updated
 - **MRS (Register $\leftarrow$ Status Register)**
 - **MSR (Status Register $\leftarrow$ Register)**
- Interrupts can be enable/disabled and modes changed, by writing to the xPSR
 - Typically a read/modify/write strategy should be used:

```

MRS r0,xPSR      ; read xPSR into r0
BIC r0,r0,#0x80  ; clear bit 7 to enable IRQ
MSR xPSR_c,r0    ; write modified value to 'c' byte only
            
```
- In User Mode, all bits can be read but only the condition flags (**_f**) can be modified

Example Source File



Summary

- Computer architecture taxonomy are in common use today
 - von Neumann vs. Harvard
 - CISC vs. RISC
 - Superscalar vs. VLIW
- The programming model is a description of the architecture relevant to instruction operation.
- ARM Instruction set
 - Load/store architecture
 - Most instructions are RISC, operate in single cycle.
 - Some multi-register operations take longer.
 - All instructions can be executed conditionally.

Quiz (1)

Q1: What is difference between Von Neumann and Harvard architecture?

Q2: What is difference between CISC and RISC?

Q3: What is difference between Microprocessor and Microcontroller architecture?

Q3: What is difference between superscalar processor and VLIW processor?

Q4: What is the difference between a big-endian and little-endian data representation?

Quiz (2)

Q5: Answer the following questions about the ARM programming model:

- a. How many general-purpose registers are there?
- b. What is the purpose of the xPSR?
- c. What is the purpose of the Z bit?
- d. Where is the program counter kept?

Quiz (3)

Q6: Write an ARM instruction which will implement each of the following:

a) $r0 = 16$

b) $r0 = r1 / 16$ (signed numbers)

c) $r1 = r2 * 3$

d) $r0 = -r0$

Q7: Explain the operation of the BL instruction, including the state of ARM registers before and after its operation.

Q8: Which data processing instructions always set the condition flags?

Quiz (4)

1. What assembler directive should start every assembler source file?
2. What instructions can be used to return from a leaf subroutine call?
3. What instructions should be used to enable or disable IRQ interrupts?
4. What instructions can be used to overcome the $\pm 32\text{MB}$ limitation of the standard ARM Branch instruction?